GenAI-VMS

The whole project, explained from zero.
A study guide and presentation companion.

GenAI-Powered Video Management System with Multimodal Search, Event Reasoning and Automated Incident Summarization

CCVR Centre of Excellence Internship · RV College of Engineering, Bengaluru
Department of MCA · Guide: Prof. Chandrani Chakravorty

Prepared for Divyansh · 1 November 2026

How to use this guide

You said you know nothing about this project. This guide assumes exactly that. It starts with the problem in everyday words, then follows **one video clip** through the whole system, then opens each part of the machine one at a time. Every part has the same four things: a **plain-English idea**, an **analogy**, a **diagram**, and a box saying **what to say when you present**.

THE ONE-MINUTE VERSION (MEMORISE THIS FIRST)

Cameras record video all day, and nobody can watch it all. **GenAI-VMS** watches it for you. It cuts each camera's video into 10-second pieces, writes a small structured "report card" for every piece (who and what is in the scene, where, doing what), notices suspicious things with simple rules, asks a vision AI model "is that really happening?", links the same event across cameras, and then uses language AI to **search** the footage in plain words, **explain** incidents in phases with a written report that cites its evidence, and **answer questions** in a chat. Everything runs on a laptop with a 4 GB graphics card, using small models, and every AI answer says which model gave it and why.

Reading order

If you have…	Read
30 minutes	Chapters 1, 2, 3 and 12 (the demo script), then the Q&A in chapter 13.
2 hours	Everything except the optional boxes marked "go deeper".
The night before	Chapter 9 (results and honesty), chapter 11 (the 5-minute and 15-minute talks), chapter 13 (Q&A), and the cheat sheet at the very end.

How the boxes work

THINK OF IT AS…

An everyday picture that makes the idea stick.

WHAT TO SAY

A sentence you can use almost word for word when presenting.

BE HONEST ABOUT…

A limit or an unfinished part. Examiners respect it when you say it first.

TRY IT

Something you can click or run to see the idea with your own eyes.

STATUS IN ONE HONEST PARAGRAPH

The system works end to end on this laptop: video in, understanding, alerts, search, reports, assistant, timeline, PDF. About three quarters of the planned backlog is done and tested (thousands of automated tests). The headline research claim "our fine-tuned model beats the untrained one" is **not proven**; we measured it honestly and it only helped on labels that came from a rule. The system has not been deployed to a real cloud server or Kubernetes cluster, no user study was held, and one benchmark run is unfinished. Chapter 10 lists everything. Knowing this list is part of presenting well.

Contents

1. **The problem and the big idea** — why this project exists
2. **The system on one page** — the map you will keep coming back to
3. **One clip's journey** — the story that ties everything together
4. **The pipeline, station by station** — cameras to alerts
5. **The GenAI machinery** — gateway, search, reasoning, assistant, reports
6. **The AI ideas you need** — a plain-English primer
7. **What the user sees, and who may do what** — interface and security
8. **Running it, keeping it healthy** — deployment, monitoring, tests
9. **What we measured** — the real numbers
10. **What is not done, and what went wrong** — the honest chapter
11. **How to present it** — 1-minute, 5-minute and 15-minute plans
12. **The live demo, step by step**
13. **Questions you will be asked** — with answers
14. **Glossary and cheat sheet**

1. The problem and the big idea

1.1 The problem

A **video management system (VMS)** is the software behind CCTV: it records cameras, shows them live, and lets you replay them. A campus or a shopping mall can have dozens of cameras recording 24 hours a day. That is thousands of hours of video, and almost all of it is boring. When something happens, a person has to scrub through hours of footage to find it.

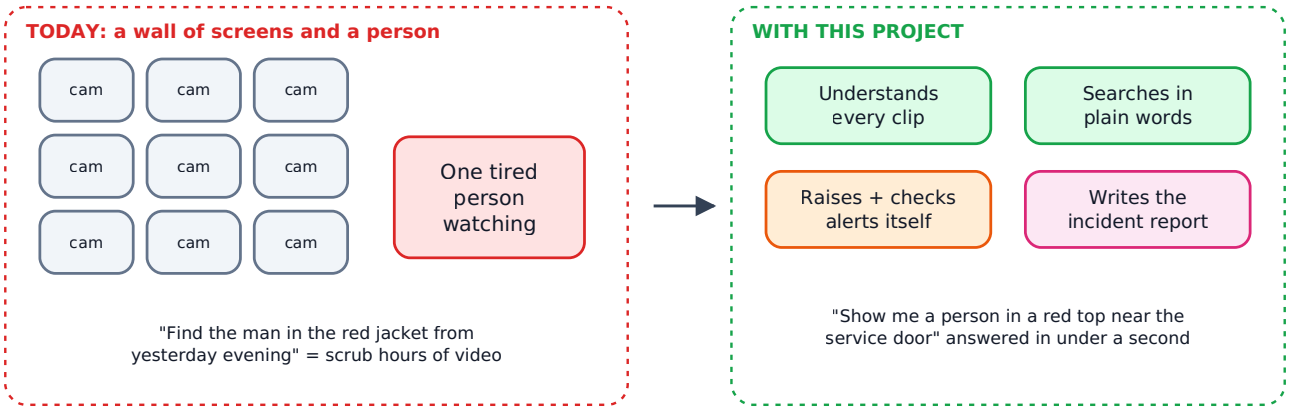


Figure 1.1 — The problem and what this project adds.

THINK OF IT AS…

A huge library where every book is a video tape and there is **no catalogue**. To find one scene you must open every tape. Our project hires a tireless librarian who watches every tape, writes a summary card for each, files the cards by meaning, and can also write you a short report on any incident — with footnotes pointing at the exact pages.

1.2 What we built, in three pillars

Pillar	The question it answers	The idea
1. Multimodal search	"Show me a person in a red top near the service door."	Describe what you want in words (or with a picture). The system finds matching moments across all cameras, without you knowing the camera or the time.
2. Event reasoning	"What actually happened, and how did it unfold?"	Detect events with rules, confirm them with a vision AI, link them across cameras, and break an incident into five phases (before, building up, the incident, after).
3. Incident summarization	"Write it up for me."	An AI writes a structured incident report (and a PDF) in which every claim cites evidence. Plus a daily security report and a chat assistant.

1.3 Where the idea comes from (the research)

The project is based on three research papers, and you should be able to name them:

1. **De Silva et al., "LLMs for Video Surveillance Applications" (IEEE TENCON 2024)** — turn video into text, then a language model can search and summarise it.
2. **Zhen et al., "MP-PVIR" (2025)** — incident reasoning: split an incident into phases, describe each phase from each camera, then write the report with an AI, checking the format.
3. **Shen et al., "Reasoning Text-to-Video Retrieval via Digital Twins" (2025)** — search by building a structured "digital twin" of each frame, then using a language model to reason over candidates.

OUR OWN CONTRIBUTION

Those papers each solve one pillar on short benchmark clips. We use **one shared representation — the digital twin — to feed all three pillars**, and we make it work on **continuous live streams** on small hardware, with a single operator-facing explanation per incident.

1.4 The constraints that shaped every decision

- **Hardware:** each of us has a laptop with a **4 GB graphics card** (RTX 3050). That is tiny for AI. So: small models (3 billion parameters), squeezed to 4 bits, and **one heavy model at a time** on the GPU.
- **Money:** zero. Free tools only; cloud AI only through free tiers and only as an option.
- **Privacy:** local-first. By default nothing leaves the laptop.
- **Time:** a few weeks, two builders. So the work was split into two **tracks** (Divyansh and Jatin) that meet at strictly defined message formats ("contracts").

WHAT TO SAY

"Surveillance generates more video than anyone can watch. We built a system that turns that video into structured knowledge, so operators can search it in plain language, get alerts that an AI has double-checked, and get incident reports that cite their evidence — all running locally on a 4 GB GPU laptop."

2. The system on one page

This is the picture to keep in your head. Four floors, top to bottom: **get the video in**, **understand it**, **notice and check things**, **explain and answer** — and a window at the bottom through which the user looks.

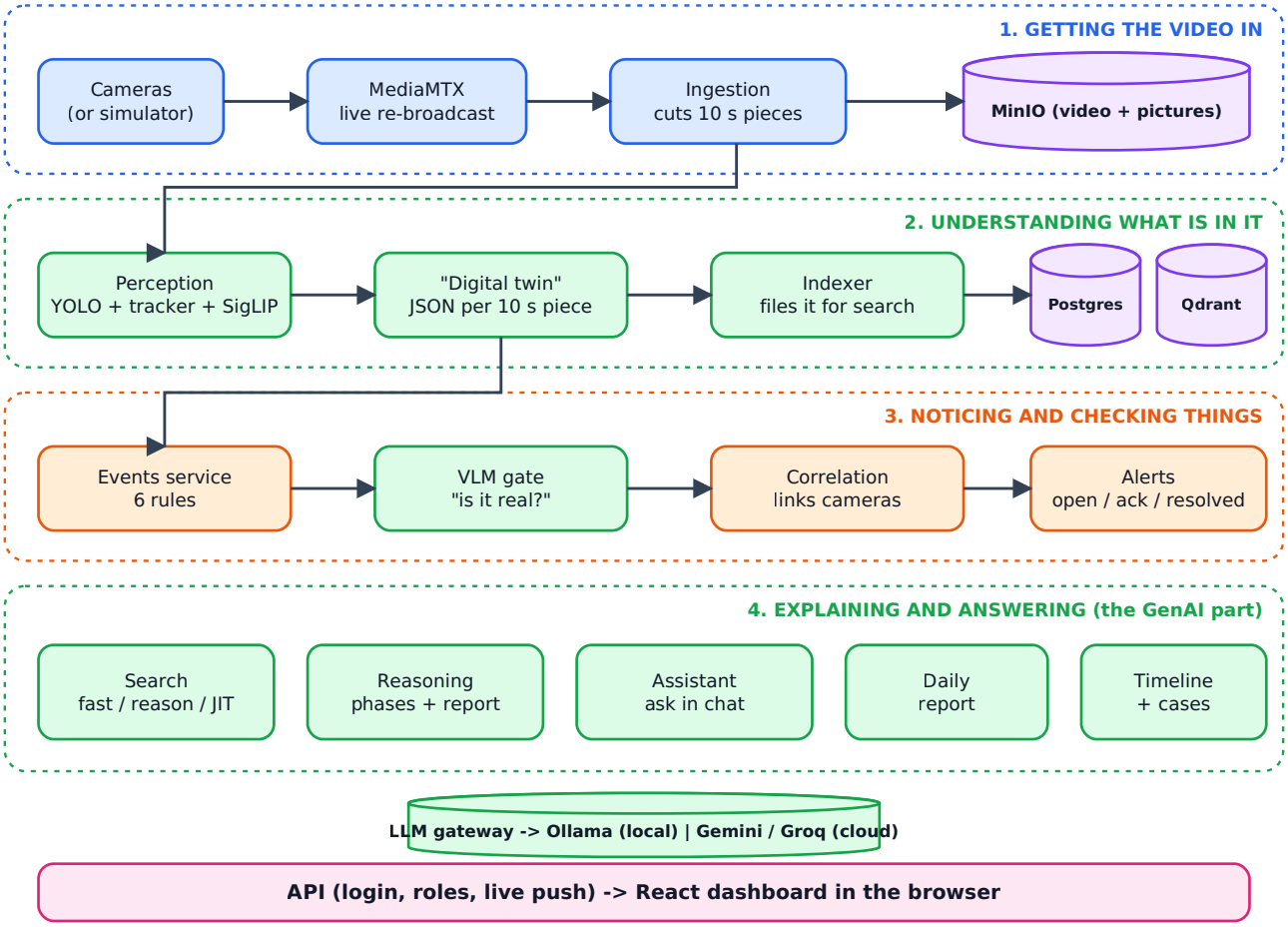


Figure 2.1 — The whole system. Arrows show what feeds what. Purple cylinders are places that hold data.

2.1 Everyday picture of each part

Part of the system	Everyday picture
Camera + MediaMTX	CCTV camera + the building's cable room
Ingestion	A mail clerk who cuts a long tape into 10-second envelopes
MinIO (object storage)	The warehouse: huge boxes (video) on shelves with addresses
Kafka	The post office: small notes with addresses, never the heavy parcels
Perception	A detective who writes a report card for every envelope
Digital twin	The report card: who/what is in the scene, where, doing what
Postgres	The filing cabinet: exact facts, searchable by column
Qdrant	A map of meaning: similar things sit close together
Events + VLM gate	A smoke alarm + a human glance that says 'real fire or toast?'
Correlation	Connecting dots: the same suspicious thing seen by two cameras
LLM gateway + GPU lease	A switchboard operator + ONE chair for the GPU: one model sits at a time
Reasoning / report	A junior analyst writing the incident report with footnotes

Figure 2.2 — Each component and its everyday twin.

2.2 The five "planes" (how the parts are grouped)

Plane	Parts	Behaviour
Media plane	MediaMTX, ingestion, MinIO	Runs all the time. **Never blocked by AI**: if every model dies, recording and playback continue.
Perception plane	perception, indexer	Near real time. Uses the GPU for the detector.
Event plane	events, correlation, alerts	Near real time. Small GPU use (the VLM gate).
GenAI plane	reasoning, search, assistant, gateway, Ollama	On demand or queued. The big GPU consumer.
Experience plane	API, React dashboard	What the operator touches.

2.3 Seven design principles (they explain almost every "why")

1. **One representation, three pillars.** The digital twin feeds search, event reasoning and summaries. Nobody re-watches raw video unless pixels are truly needed.
2. **Cheap filter first, expensive reasoning second.** Rules before the vision AI; embeddings before the language-model rerank; correlation before phase reasoning.
3. **Metadata on Kafka, pixels in object storage.** Messages carry links, never video.
4. **Local-first, cloud-switchable.** Every AI call goes through one gateway chosen by task name and profile.

5. **Contract-first parallel work.** Divyansh and Jatin build against agreed message formats, not each other's unfinished code.
6. **Evidence or it didn't happen.** Every generated claim cites evidence ids.
7. **Degrade, don't die.** If a model is down, features fall back; recording never stops.

WHAT TO SAY

"If I remember one slogan: cheap filter first, expensive reasoning second. It is why a 4 GB laptop can run this."

3. One clip's journey

Abstract diagrams are hard to remember; a story is easy. Follow one 10-second piece of video from **camera cam03**, where a person lingers near a restricted door.

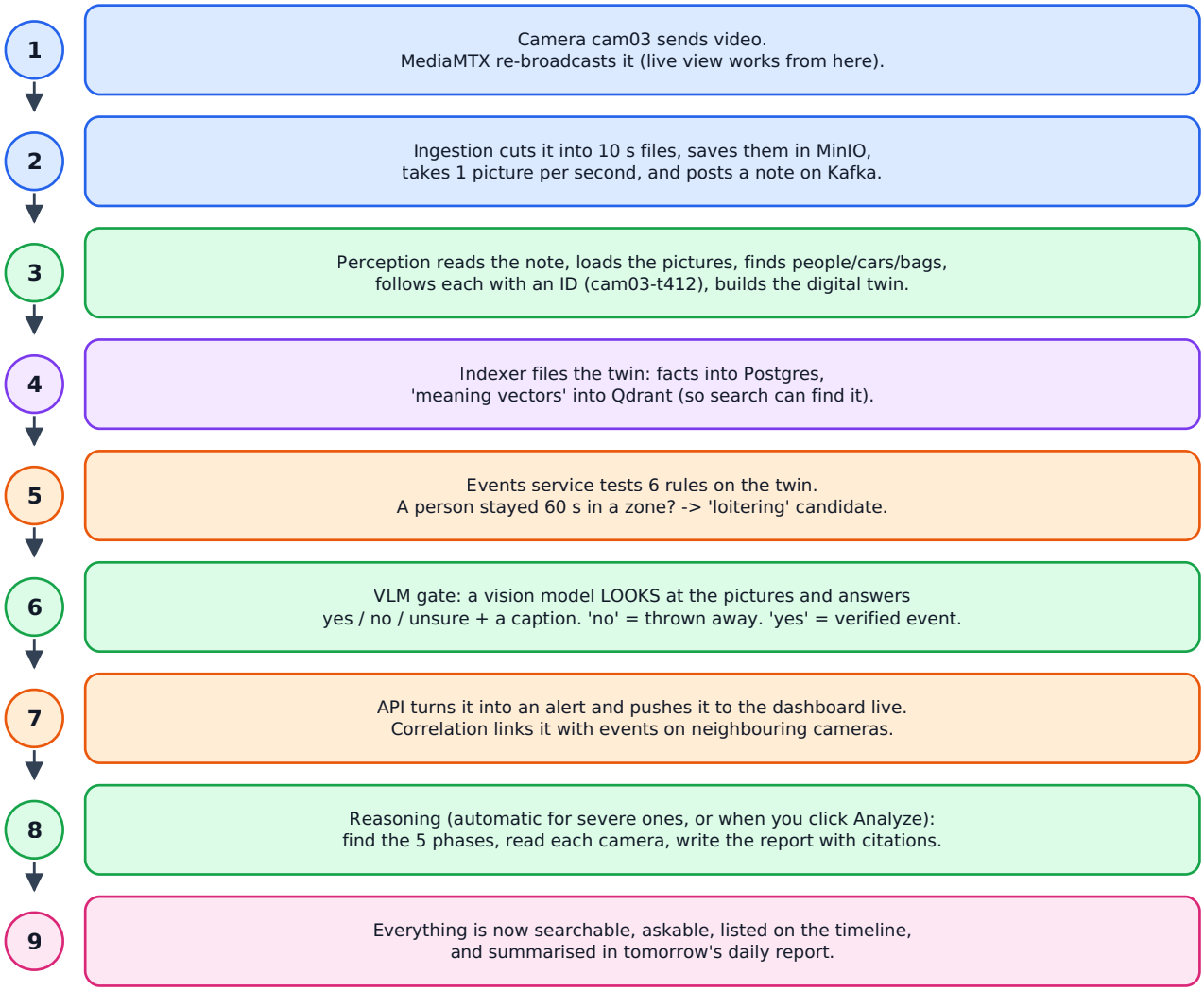


Figure 3.1 — Nine steps from camera to answer.

3.1 The same story as a timeline

When	What happens	Who does it	What is produced
10:15:00	cam03 streams video	MediaMTX	a live stream everyone can watch
10:15:10	The 10 s piece is closed and saved	ingestion	a `.ts` file in MinIO, 10 keyframes, a Kafka note
10:15:12 +	Objects found and followed	perception	the digital twin (JSON)
10:15:13 +	Facts and vectors filed	indexer	rows in Postgres, points in Qdrant
10:16:15	Person has been in the zone for 60 s	events (rules)	a loitering candidate

+ a few seconds	"Is this real?"	VLM gate	verdict **yes** + caption, so a verified event
immediately	Operator sees a card	API + dashboard	an **open alert** pushed live
when the group closes	Linked to events on cam02	correlation	a correlation group
on demand	Operator clicks **Analyze**	reasoning	a 5-phase incident report with citations and a PDF

WHAT TO SAY

"Let me follow one 10-second clip through the system..." — then use the nine steps. Telling it as a story beats listing components.

4. The pipeline, station by station

Now we open each box of the big picture. The order is the order the video travels. Remember the rule that explains the design: **heavy things (video, pictures) sit in storage; only small notes with links travel between services.**

4.1 Cameras and MediaMTX — getting the video in

Cameras speak a protocol called **RTSP** (a way to stream video over a network). We do not have a building full of cameras, so a **camera simulator** plays recorded datasets as if they were live RTSP cameras (it uses `ffmpeg`). We replayed real footage from public research datasets (MEVA bus-station cameras, UCF-Crime clips) this way.

MediaMTX is the media server in the middle. It receives each stream and re-publishes it two ways: **WebRTC** (very low delay, used for the live wall) and **HLS** (chopped into small files, used for playback in a browser).

> **THINK OF IT AS…**
> A TV broadcasting hub. Cameras are the studios; MediaMTX is the control room that re-broadcasts each channel in two formats: live (WebRTC) and on-demand (HLS).

4.2 Ingestion — cutting the tape into envelopes

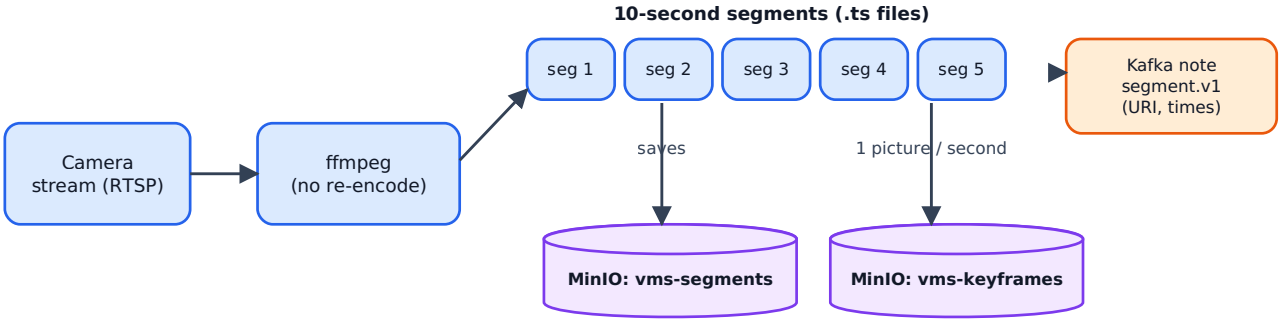


Figure 4.1 — Ingestion: one worker per camera cuts the stream into 10-second pieces.

1. There is **one asynchronous worker per enabled camera**. The list of cameras comes from the API (and falls back to a config file); it is refreshed every 60 seconds, so adding a camera needs no restart.
2. `ffmpeg` copies the stream into **10-second MPEG-TS files without re-encoding** (so it costs almost no CPU).
3. Each file is uploaded to the MinIO bucket `vms-segments`; **one picture per second** (a keyframe) goes to `vms-keyframes`.
4. It publishes a small Kafka message `segment.v1`: camera, segment id, start and end time in UTC, and the storage link.
5. If the stream drops it reconnects with exponential waiting (1, 2, 4… up to 30 s) and the next message is flagged `gap_before = true`, so nobody mistakes a gap for silence.

> **BE HONEST ABOUT…**
> The 30-minute, 6-camera "no missing segment" stress test was written but never run as specified. Shorter real runs and the "restart workers mid-stream" chaos test passed (chapter 9).

4.3 Kafka — the post office

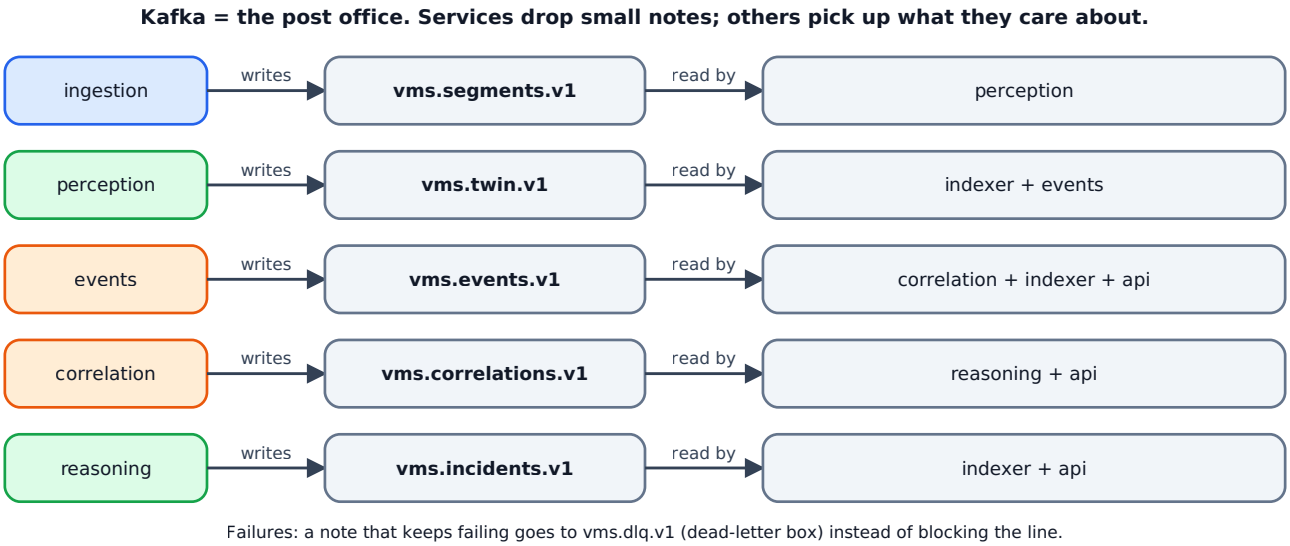


Figure 4.2 — Kafka topics: who writes, who reads.

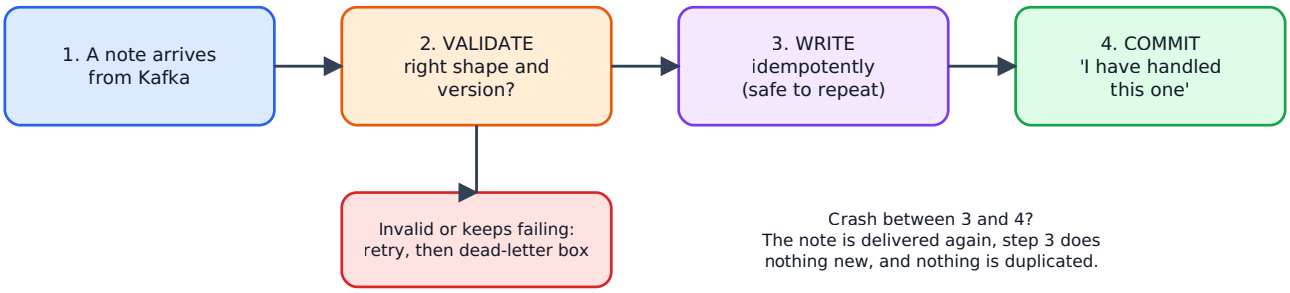


Figure 4.2b — What every consumer does with every message.

Kafka is a message system. A **producer** writes a message to a **topic**; any number of **consumers** read it at their own speed. Because the queue sits between services, a slow GPU service never slows the one before it, and if a service crashes it simply resumes where it stopped.

- **Messages carry links, not video** (and must stay under 1 MB). This was decision D-02.
- Every topic name ends in a version (`.v1`). A message format may only gain optional fields; anything else needs a new version and a new topic. These formats are the **contracts** between Divyansh's and Jatin's code; each has a Python class (Pydantic) and a sample file (fixture).
- Consumers follow one rule: **validate, write idempotently, then commit**. "Idempotent" means doing it twice gives the same result as doing it once, so a replayed message cannot create duplicates.

- A message that keeps failing goes to the **dead-letter topic** (`vms.dlq.v1`) instead of blocking everything. There is a tool to inspect and replay it.

> **THINK OF IT AS…**
> A post office with labelled pigeonholes. The clerk who cuts the tape drops a note into the "segments" hole saying "parcel at shelf 12, camera 3, 10:15:00". The detective collects notes from that hole when ready; the clerk does not wait.

4.4 Perception — writing the report card (the digital twin)

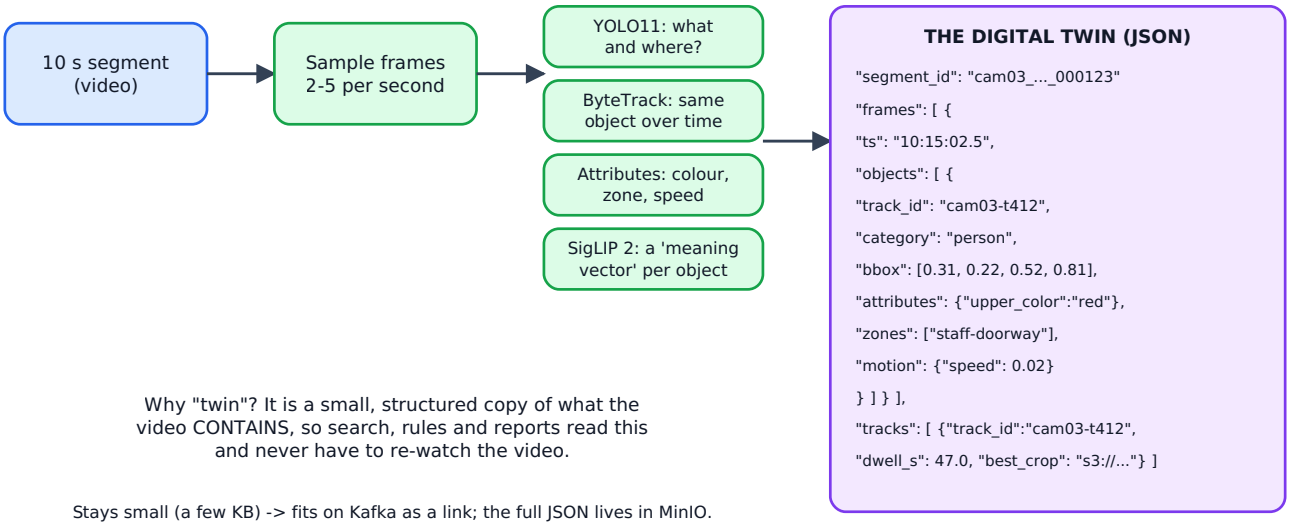


Figure 4.3 — From a video segment to a digital twin.

Perception is the first AI step. For every 10-second segment it:

1. **Samples frames** (2 to 5 per second) from the video.
2. **Detects objects** with **YOLO11**: what is in the picture and where (a bounding box), for people, cars, bags and so on.
3. **Tracks them** with **ByteTrack**: the same person in consecutive frames gets the same id (like `cam03-t412`), even across segment boundaries. The tracker state is checkpointed in Redis so a restart continues the numbering.
4. Adds **attributes** (colour of clothes), the **zone** the object is in (polygons an admin draws on each camera: e.g. "staff doorway"), and **motion** (speed, direction).
5. Computes a **SigLIP 2 embedding**: a list of numbers that captures what the object looks like, used later for "find things like this".
6. Writes it all as the **digital twin** JSON, stores it in MinIO and announces it on Kafka.

> **THINK OF IT AS…**
> A detective who watches each 10-second envelope and fills in a standard form: "3 people, 1 backpack; person #412 in a red top stayed 47 seconds in the staff doorway". Later, everyone reads the **form**, nobody re-watches the tape.

WHY A "LIGHTWEIGHT" TWIN?

The research paper builds a heavy twin for every frame (segmentation plus depth). That is impossible live on 4 GB. We keep a small twin (boxes, tracks, attributes, zones, vectors) and **draw detailed object outlines (masks) only on demand** for search results. Measured: both perception models together use only about **966 MB** of GPU memory.

4.5 The storage trio (plus Redis)

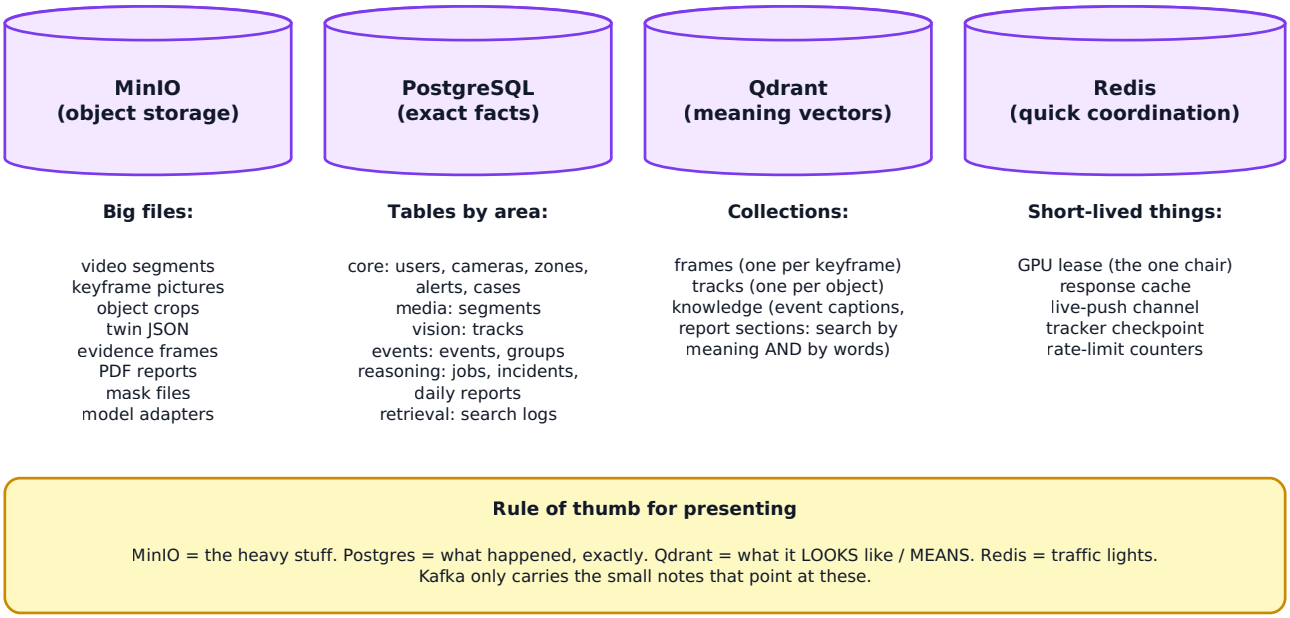


Figure 4.4 — What lives where.

Beginners often ask "why four databases?" Because each is good at one job:

- **MinIO** speaks the same language as Amazon S3. It is for big files and gives each a link. Locally it is free; in the cloud it can be swapped for real S3.
- **PostgreSQL** is the classic relational database: exact facts you can filter and join ("all high-severity events on cam02 yesterday").
- **Qdrant** is a **vector database**: it stores the "meaning vectors" and finds the closest ones. It also supports hybrid search (by meaning and by exact words).
- **Redis** is a fast scratchpad: the GPU lease, response cache, live-push channel, rate limits.

4.6 The indexer — filing the report card

The indexer listens to the twin, event and incident topics and files everything: segments, tracks and counts into Postgres; frame and track vectors into Qdrant; event captions and incident-report sections into the `knowledge` collection (two search indexes at once: a **dense** one for meaning and a **BM25** one for exact words). It is idempotent, so replaying a message changes nothing.

4.7 The events service — rules plus a second opinion

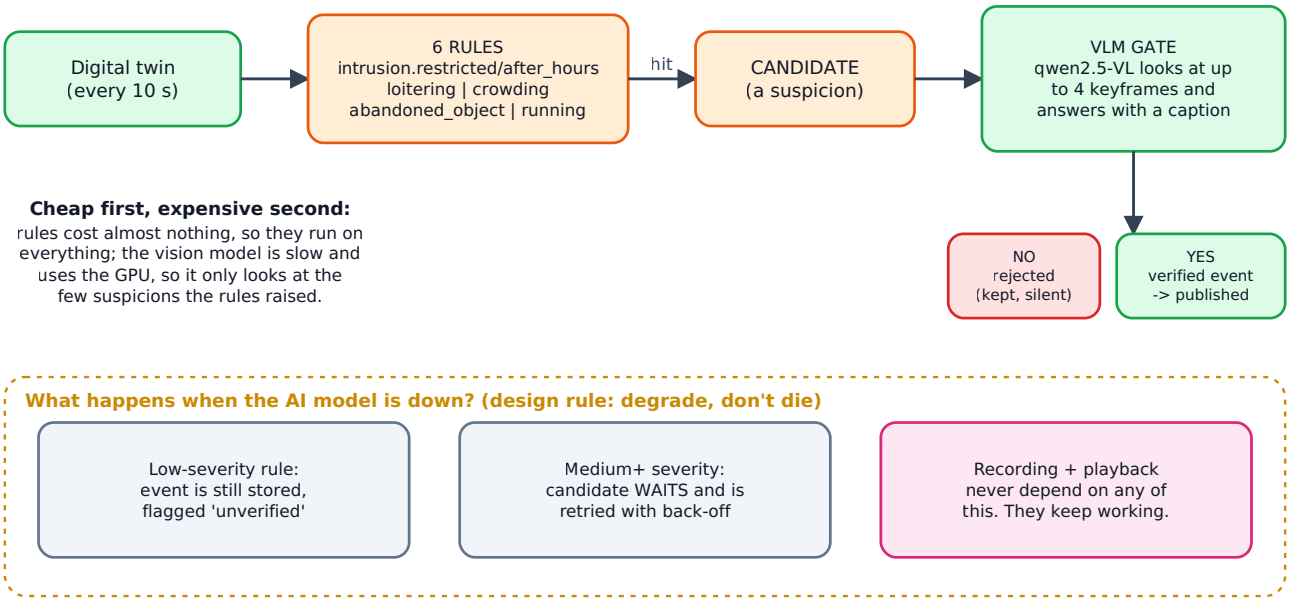


Figure 4.5 — Rules suspect; the vision model confirms; low-quality alarms die quietly.

The six rules (config/rules.yaml)

Rule	Fires when…	Severity
`intrusion.restricted`	a person/vehicle is seen in a zone marked restricted for at least 2 sampled frames	high
`intrusion.after_hours`	a person is seen in a zone outside its allowed hours (zones can have a schedule)	high
`loitering`	a person stays in a zone for 60 s or more	medium
`crowding`	more than 8 people in a zone for 20 s	medium
`abandoned_object`	a bag stays still 30 s and its owner has been away 20 s	high
`running`	a person moves faster than a speed threshold (0.35 frame-widths per second) in at least 3 samples	low

Rules look at consecutive twins, so one long episode becomes **one** candidate, not a hundred (the rule engine folds consecutive hits and debounces them). Thresholds live in a config file, not in code.

The VLM gate

A **VLM** (vision-language model) is an AI that can look at pictures and talk about them. Our gate sends up to 4 keyframes of the candidate to `qwen2.5-VL 3B` (running locally through Ollama) and asks for structured JSON: a **verdict** (yes / no / unsure), a confidence, and a one-sentence **caption**. The result is one of:

- **verified** — published as an event and becomes an alert (if severe enough);
- **rejected** — stored with its reason (useful for measuring the gate), never published;

- **skipped** — the model could not be asked (or the rule has verification off); published but flagged unverified.

A REAL RESULT WORTH QUOTING

On a replay of real bus-station footage (MEVA), the rules raised **49 candidates** and the gate judged all 49. It **verified 30 of 30 loitering** candidates in the waiting room, but **rejected all 9** "restricted-doorway intrusion" candidates (people were just walking past) and rejected 2 of 4 abandoned-object candidates. That is the gate earning its keep: it removes false alarms that rules alone would have sent to the operator.

BE HONEST ABOUT…

Those figures come from one 10-minute replay with zones drawn by hand. The gate's precision and recall against labelled clips was **not** measured (that needs labelled data). Also, 31 of the 37 verified alerts were waiting-room loitering: a rules-tuning problem, not an AI problem.

4.8 Alerts

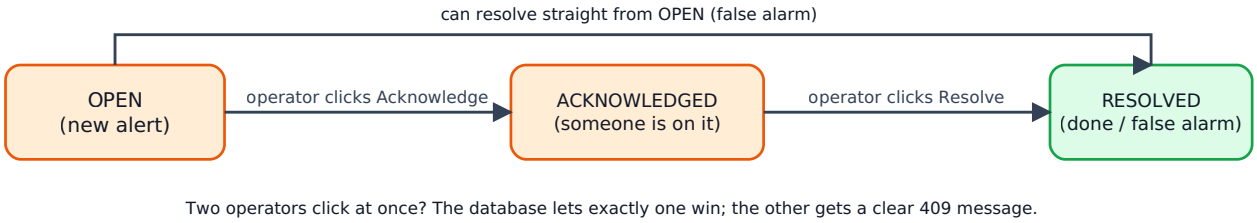


Figure 4.6 — The life of an alert.

The API consumes verified events and creates **one alert per event** (severity at least medium by default; lower ones are stored and searchable but do not interrupt anyone). Each alert is pushed to every dashboard through a **WebSocket** the moment it is created. Notification channels (dashboard, email, Telegram) are plugins; only the dashboard is on, by team decision. Actions are **audited** (who acknowledged what, with their note).

4.9 Correlation — connecting the dots across cameras

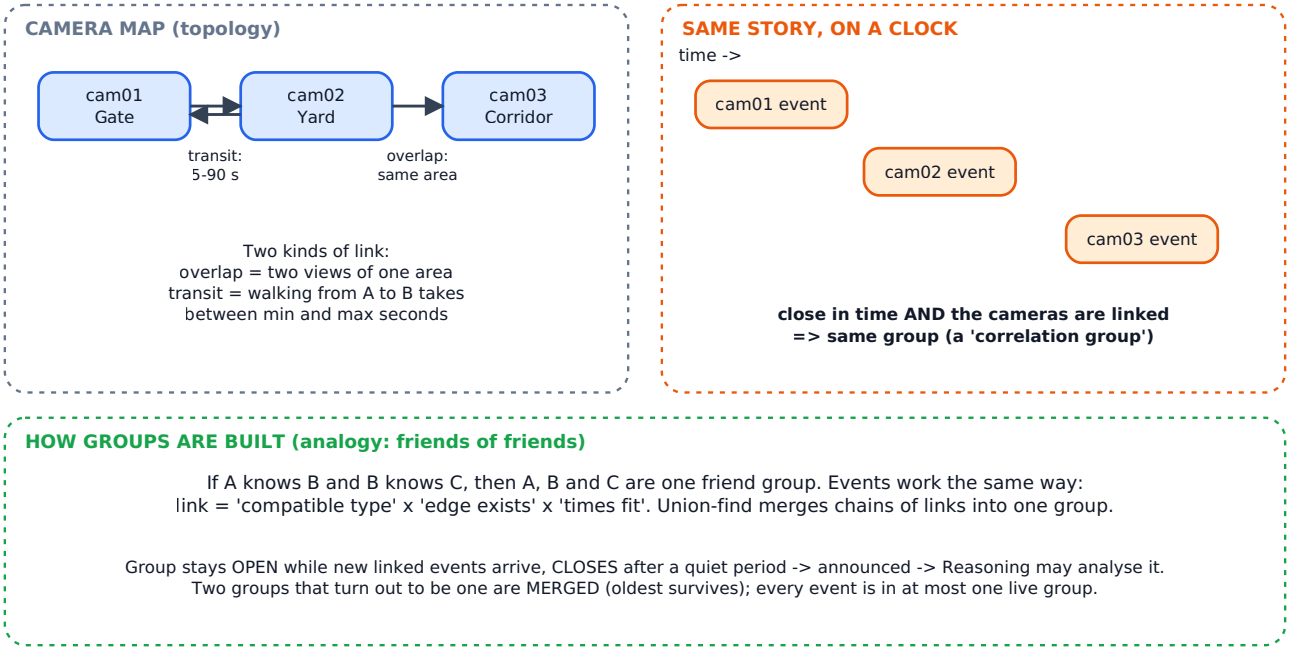


Figure 4.7 — Camera topology, time, and grouping.

A single camera sees only part of the story. The **camera topology** is a small map an admin draws: which cameras overlap (two views of one area) and which are linked by **transit** (walking from A to B takes between a minimum and maximum number of seconds). The correlation service links events on **different** cameras if (1) their event types are compatible (a matrix in `config/correlation.yaml`), (2) the cameras are connected, and (3) the times fit once a tolerance is added. Chains of links are merged into groups with **union-find**.

> **THINK OF IT AS…**
> Friend groups. If A knows B and B knows C, then A, B and C are one group, even if A never met C. Events on different cameras become one "story" the same way.

A group stays **open** while linked events keep arriving and **closes** after a quiet period, which is the signal for the reasoning service. If two groups turn out to be one, they are **merged**: the oldest survives and absorbs the rest. A hard rule: an event is in at most one live group.

> **BE HONEST ABOUT…**
> Correlation uses **time and the camera map only**. It does not recognise that the same person appears on two cameras (person re-identification is future work, decision D-10). So in busy scenes it can over- or under-link.

5. The GenAI machinery

Chapter 4 was the plumbing that produces structured facts. This chapter is where the **generative AI** comes in: one gateway that every model call passes through, and four features built on it.

5.1 The LLM gateway — one switchboard for every model

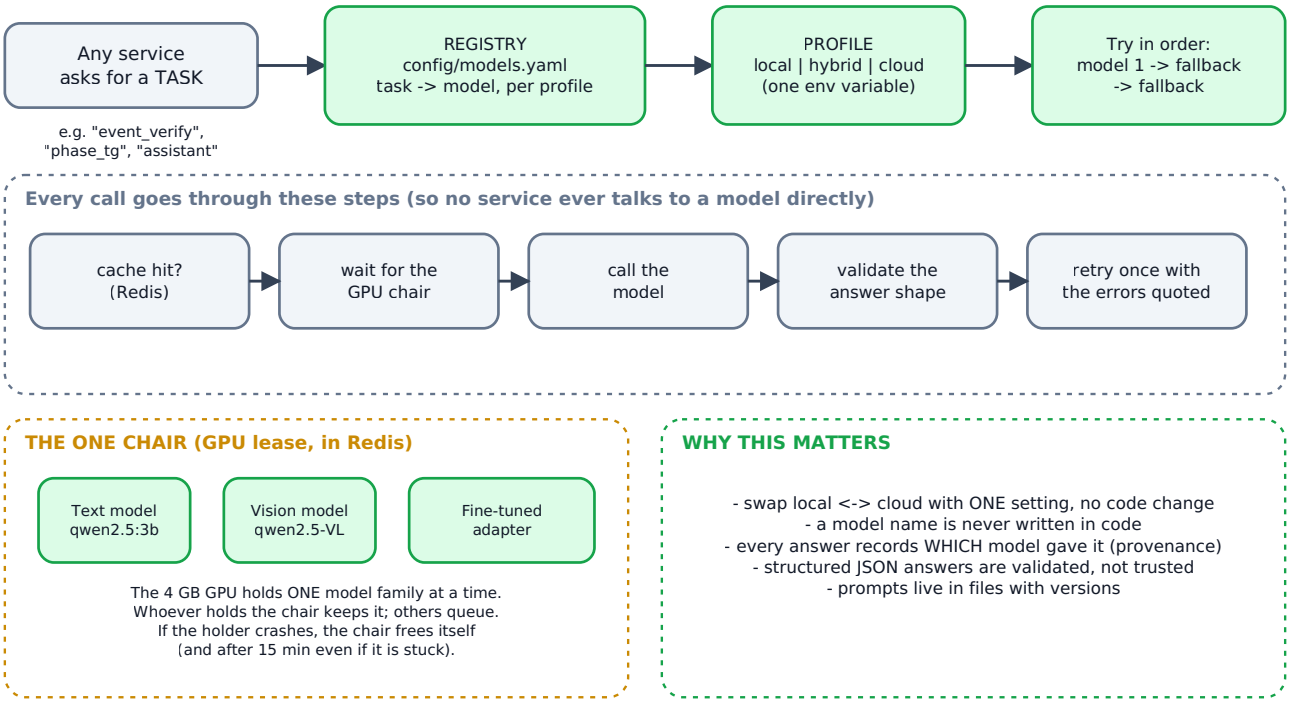


Figure 5.1 — The gateway and the GPU lease.

No service ever says "call model X". It says **"do task `event_verify`"**. The gateway looks the task up in `config/models.yaml`, finds the model for the active **profile**, and handles everything around the call.

Profile	Meaning
`local`	Everything on this machine through **Ollama**: vision tasks on `qwen2.5vl:3b`, text tasks on `qwen2.5:3b`. Default for development.
`hybrid`	Vision stays local; text reasoning (query planning, reranking, reports, assistant, daily narrative) goes to cloud models (Gemini / Groq free tiers).
`cloud`	Everything in the cloud (for a server with no GPU).

The tasks are: `event_verify`, `jit_vqa`, `query_decompose`, `rerank`, `phase_tg`, `phase_vr`, `incident_synthesis`, `assistant`, `daily_narrative` (and an evaluation judge).

What the gateway does on every call

1. **Cache check** (Redis): an identical question gets the stored answer. Only for tasks where that is safe.
2. **Wait for the GPU chair** (the lease, below).

3. **Call the model** with a timeout. If it fails, try the **next model in the fallback list**.
4. **Validate the answer shape.** We ask for JSON that must fit a Python schema (Pydantic). If it does not fit, the gateway shows the model its own mistakes and asks again (up to 2 more times).
5. **Record provenance:** which provider and model answered, how many tries, whether a fallback was used. Reports store this.

> **THINK OF IT AS…**
> A switchboard operator. You ask for "the doctor" (a task); the operator knows which doctor is on duty today (the profile), dials them, and if nobody picks up, tries the next one. And there is **only one consulting room** (the GPU): one doctor at a time.

The GPU lease (ADR-007)

A 4 GB card cannot hold the text model and the vision model at once without thrashing. A **lease** in Redis says which model family currently holds the GPU. Calls of the same family share it; a call of a different family **waits its turn** (and goes first in line so it is not starved). The lease expires if the holder crashes. We recently added a safety net: even a stuck holder stops being renewed after 15 minutes. (We found that bug the hard way during a benchmark; see chapter 10.)

> **WHAT TO SAY**
> "Models are configuration, not code. Switching from local to cloud is one environment variable, and every answer records which model produced it."

5.2 Search — "find it in plain words"

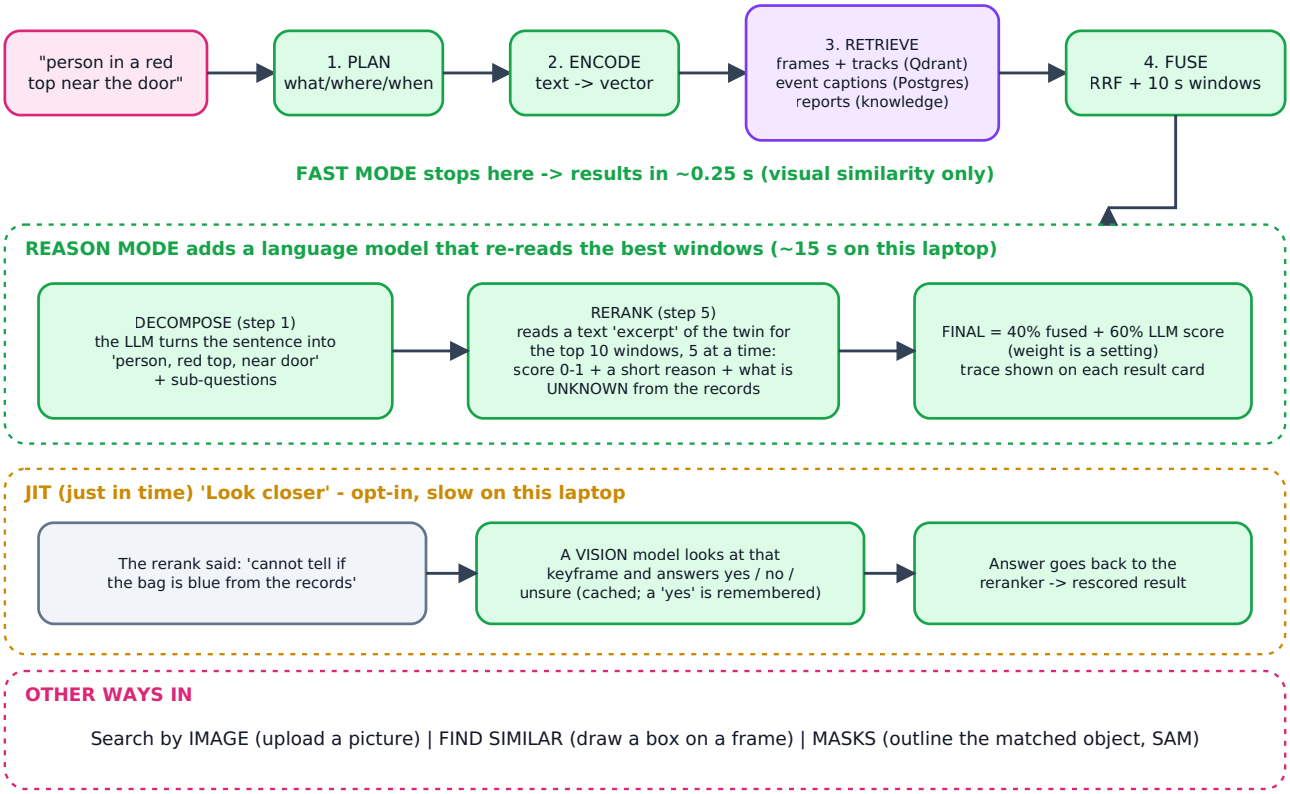


Figure 5.2 — The search pipeline: fast mode, reason mode, and JIT.

Walking through one query

Query: "a person in a red top near the service door"

1. **Plan.** In fast mode a keyword plan is enough (category = person, colour = red, zone = service door, time words like "yesterday"). In reason mode a language model **decomposes** the sentence into entities, relations, and sub-questions; relative times are resolved with the current time and the site's timezone.
2. **Encode.** SigLIP 2 turns the text into a vector on the CPU (so the GPU stays free).
3. **Retrieve.** The vector is compared with picture vectors of **frames** and **tracks** in Qdrant, with hard filters (camera, time, object type); event captions are searched in Postgres; the `knowledge` index is searched by meaning and by words. Colour and zone boost a result rather than filter it (the colour naming is rough).
4. **Fuse. Reciprocal Rank Fusion** merges the ranked lists (figure 5.3); hits within 10 seconds on one camera become one **window**; the best 30 windows are kept.
5. **Rerank (reason mode).** For the top 10 windows the system builds a short text **excerpt of the digital twin** ("a person, red top, in service-door zone, seen 14 times, speed 0.02") and asks a language model, five windows per call, for a **score 0 to 1**, a one-sentence **reason**, and the sub-questions it **cannot answer from records**. Final score = 40% fused + 60% model score.
6. **JIT refinement (optional "Look closer").** If the reranker said "cannot tell whether the bag is blue", a **vision model looks at that keyframe** and answers yes/no/unsure. Answers are cached, fed back for rescoring, and a "yes" is also written into the knowledge index so a later search can find the footage without asking again.

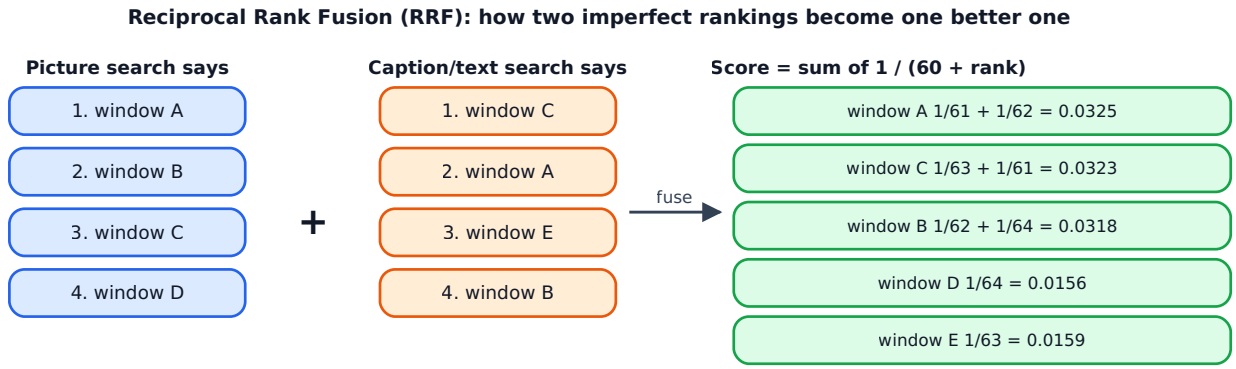


Figure 5.3 — Reciprocal Rank Fusion with real arithmetic.

Other ways to search

- **Search by image:** upload a photo; it is embedded and matched against tracked objects.
- **Find similar:** draw a box on any frame (a result, an event keyframe, or the paused playback frame). The server cuts that part out of the stored keyframe and searches with it. built; browser check pending
- **Masks:** for a result, SAM 2.1-tiny draws the outline of the matched object over the picture (computed once, cached).

THINK OF IT AS…

Fast mode is a good librarian grabbing the books whose covers look right. Reason mode is the librarian then **reading the blurbs** of the ten best candidates and ranking them by how well they really fit. JIT is going back to **open the book at one page** to check a detail.

NUMBERS TO KNOW

Fast mode: about **0.22 s** median (0.25 s at the 95th percentile). Reason mode: about **15.5 s** on this laptop (almost all of it the rerank). Cold object masks: 1.3 s; cached: 9 ms.

BE HONEST ABOUT…

Search separates what is in the picture (phone, laptop, bike, trunk) far better than what is happening (turning left, handing over an object). A single frame cannot show motion. And a clean 4-way ablation (embedding only, +filters, +rerank, +JIT) is built but the final clean run was not completed; use the earlier results in chapter 9.

5.3 Reasoning — phases, evidence and the incident report

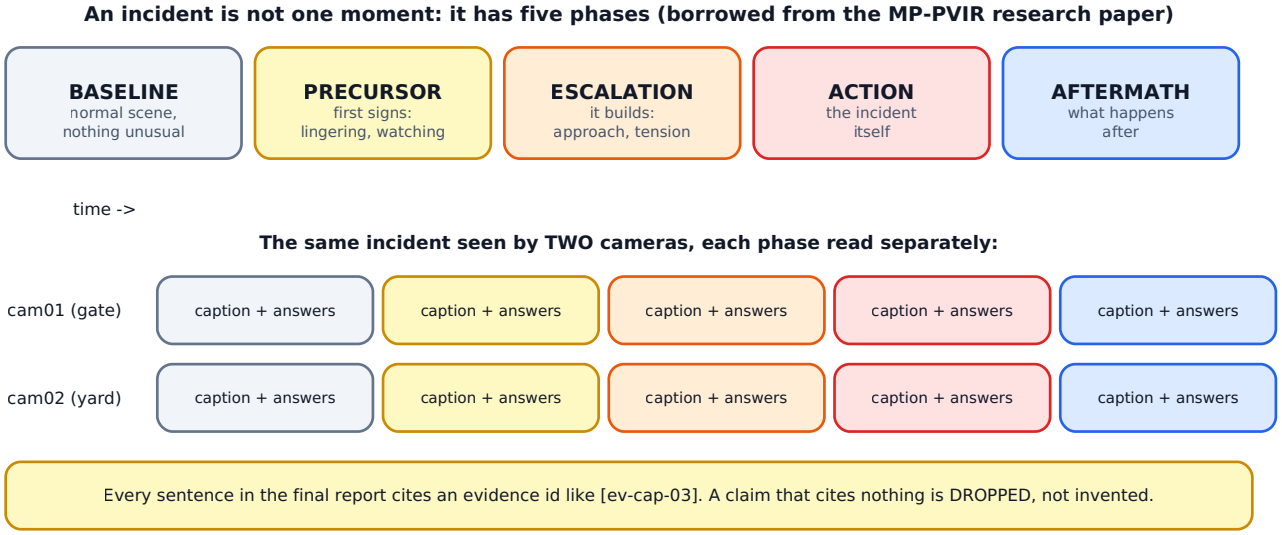


Figure 5.4 — Five phases, read separately for each camera.

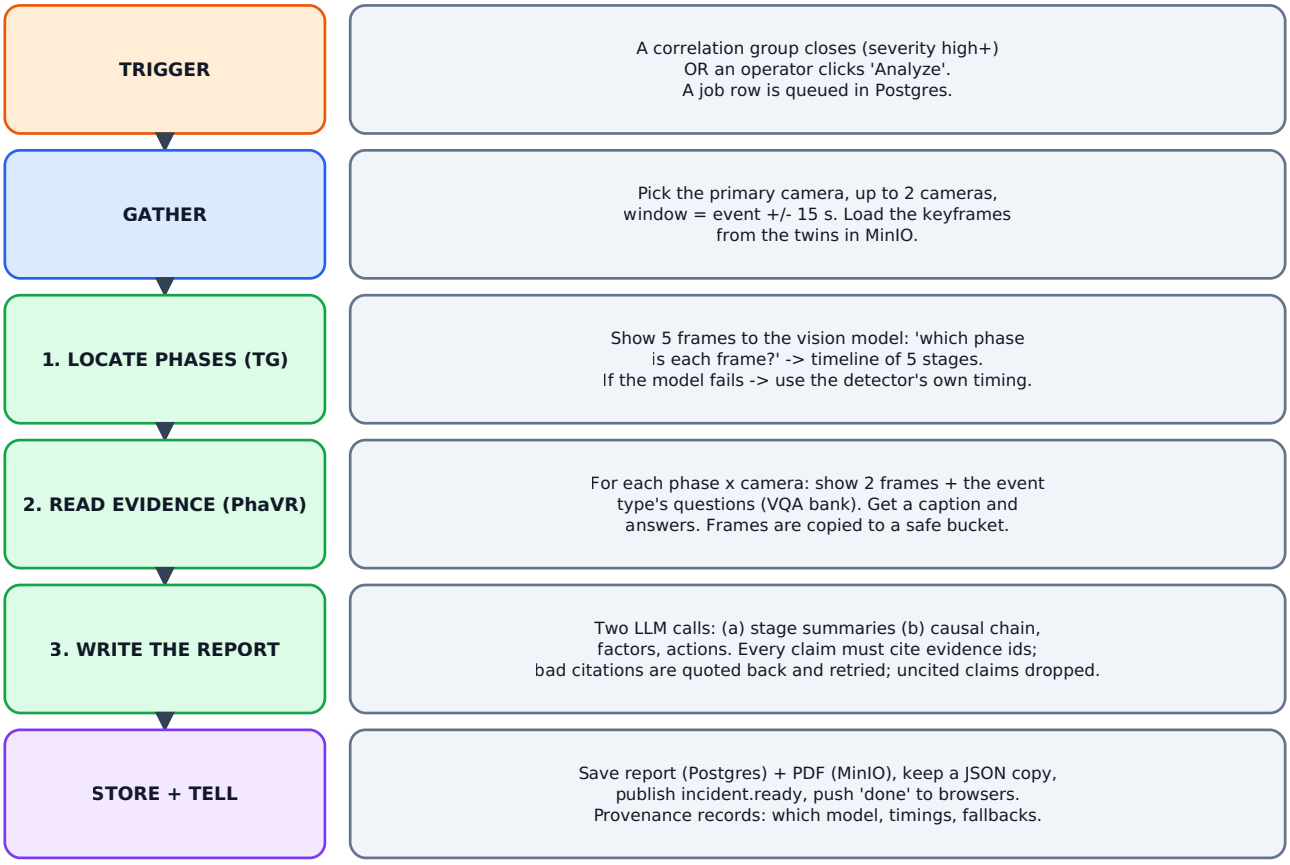


Figure 5.5 — One reasoning job from trigger to stored report.

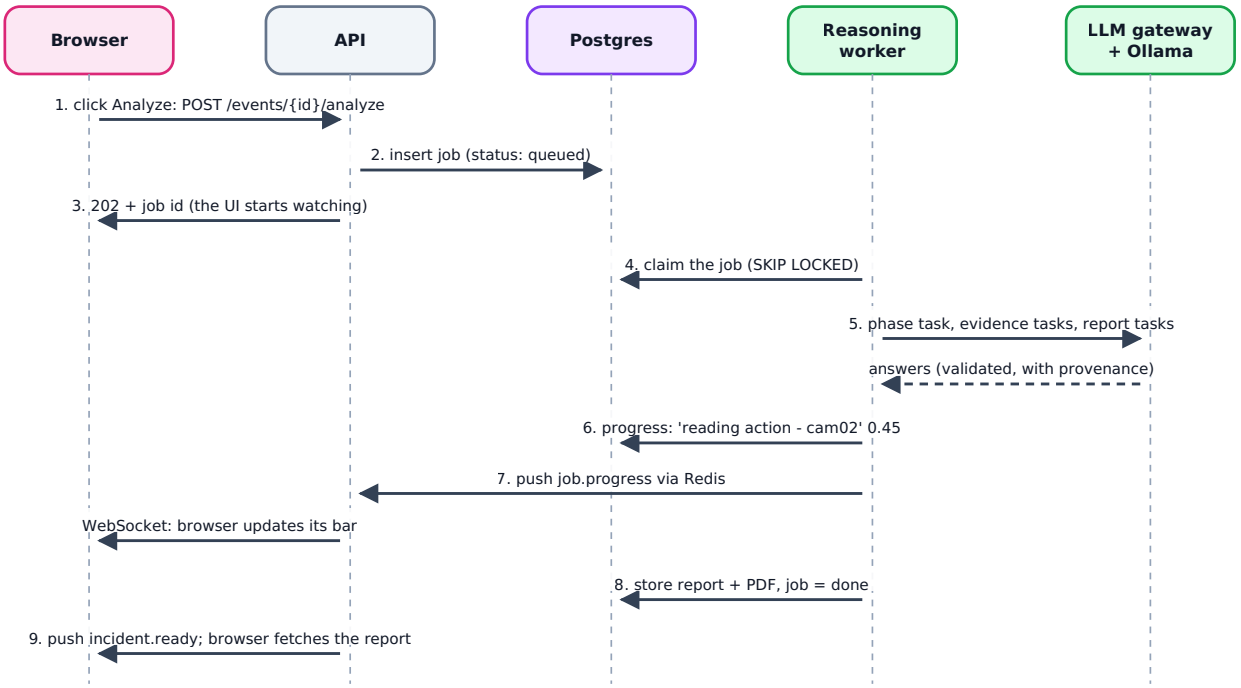


Figure 5.5b — What happens, in order, when an operator clicks Analyze (who talks to whom).

The steps in plain words

1. **A job is queued** in a Postgres table (claimed with `FOR UPDATE SKIP LOCKED`, so two workers never take the same job; a worker that dies loses its lease and the job is claimed again).
2. **Phase location (TG = temporal grounding).** Five frames from the primary camera are shown to the vision model: "which of the five phases is each frame?" The answers become spans on a timeline; impossible answers (going backwards in time) are repaired or rejected, and then the detector's own timing is used instead.
3. **Evidence (PhaVR = phase-specific captioning and VQA).** For each phase and each camera, two frames and a few event-specific questions from a **question bank** (e.g. "is the area lit?") are shown; the answers are checked against the allowed vocabulary and become **evidence items with stable ids** such as `ev-cap-03`. The cited frames are copied to a separate bucket so they outlive the keyframe retention.
4. **Report.** Two schema-checked LLM calls write stage summaries, then the causal chain, factors and recommended actions. Every claim must cite evidence ids that exist; bad citations are quoted back to the model for a retry; a claim that still cites nothing is **dropped and counted**. A tolerance exists for near-misses like `[ev-cap-1]`.
5. **Store and announce.** The report goes into Postgres and a PDF into MinIO; a JSON copy is kept; browsers get a live "report ready" push. **Provenance** says which model located the phases ("adapter", "zero-shot" or "rules"), per-step timings, and how many views were skipped.

THINK OF IT AS…

A junior analyst: first marks on a timeline "the trouble started here", then sits with each camera's footage for each stage and takes notes with photo references, then writes the report with **footnotes** to those notes. A supervisor (the checker) strikes out any sentence without a footnote.

THE ONE-LINE PRINCIPLE

Evidence or it didn't happen. That is how we handle AI "hallucination": the model is not allowed to assert something that it cannot point at.

BE HONEST ABOUT…

The default phase locator is the **untrained** vision model, and the model that reads each phase is also untrained: the second fine-tuned adapter (PhaVR) was never trained because it needs caption annotations nobody had time to make. If any model fails, the system falls back to the detector's own timing, and says so in the report's provenance.

5.4 The assistant — ask in chat

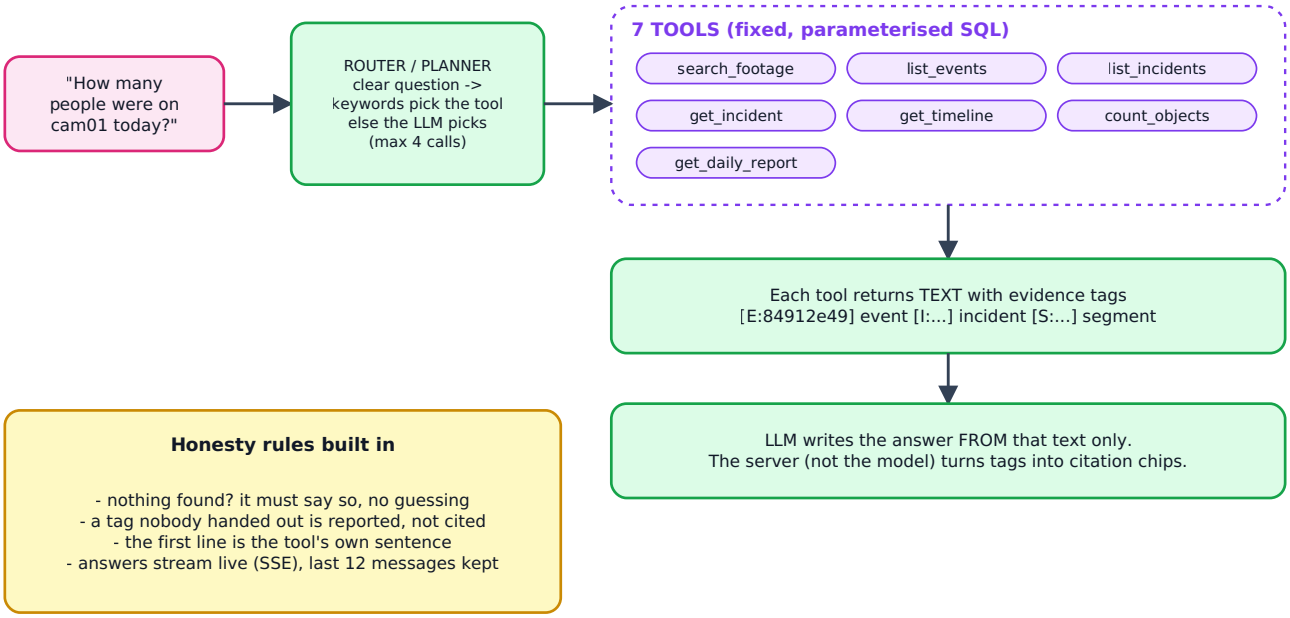


Figure 5.6 — How one question becomes a cited answer.

Examples the assistant handles: "What happened today?", "How many people were on cam01 yesterday?", "List high-severity incidents", "Show me the daily security report for yesterday", "Find a white car near the gate". A small model computes dates badly, so questions with clear keywords are **routed** to the right tool without asking the model; the rest go to a **planner** that returns a JSON action (at most 4 lookups). The conversation keeps the last 12 messages and a rolling summary.

THINK OF IT AS…

A receptionist with seven filing cabinets (the tools). You ask a question; the receptionist pulls the right folder, **reads you what is in it**, and adds only a short explanation. If the folder is empty, the receptionist says "I found nothing", never "I think it was probably...".

WHAT TO SAY

"The assistant never writes database queries. It can only call seven fixed, parameterised tools. And every answer is built from what those tools returned, with citation chips you can click."

5.5 Daily reports, the timeline and cases

- **Daily security report.** A fixed set of SQL queries produces the day's **facts** (counts by type, camera, hour; incidents; alert response times). An LLM writes a short narrative from those figures; a **post-check demands that every number in the text appears in the facts** (one retry, then a template text, and the report says which was used). It is generated automatically after 06:00 site time for the previous day, or on demand for up to 7 days, and exported as a PDF.
- **Investigation timeline.** One lane per camera showing events and incident reports across a period; events linked across cameras are outlined and shown on a "Linked" row. Drag across a lane to pick a period, then zoom, open the recording, play several cameras **in sync from one playhead** (measured drift well under 300 ms), save to a case, or **search in that range**.
- **Cases.** Bookmark events, incident reports, footage moments and searches into an investigation with notes.

6. The AI ideas you need

You do not have to be an AI expert. You need about a dozen ideas, each explained with a picture in your head.

Idea	In one sentence	Everyday picture
Object detection (YOLO)	A model that looks at a picture and draws boxes around things with labels ("person", "car", "backpack").	A spot-the-objects robot that circles things in a photo.
Tracking (ByteTrack)	Gives the same object the same id in consecutive frames.	Putting a name tag on every person in a crowd and following the tags.
Embedding	A list of numbers (a vector) that captures the meaning of a picture or sentence, so similar things get similar numbers.	GPS coordinates on a map of meaning: "red jacket" and "scarlet coat" are neighbours; "banana" is far away.
Vector search	Find the stored vectors closest to the query vector.	"Who lives nearest to this point on the map?"
SigLIP 2	A model that makes embeddings for both text and images **in the same space**, so a sentence can find a picture.	A translator that puts words and photos on the same map.
Hybrid search + RRF	Combine search by meaning (dense) with search by exact words (BM25), then merge the rankings.	Asking two friends for recommendations and trusting the places both name.
LLM / VLM	Large language model = reads and writes text. Vision-language model = also sees pictures.	A reader vs. a reader with eyes.
Token & context window	Models read in word-pieces (tokens). The context window is how many fit at once. In our setup one image costs about a thousand tokens.	The model's short-term memory.
Quantization (4-bit)	Storing a model's numbers with fewer bits so it fits in memory.	Saving a photo as a smaller JPEG: a little quality lost, a lot of space saved.
Zero-shot vs. fine-tuning	Zero-shot = ask a general model straight away. Fine-tuning = further train it on your examples.	Asking a bright stranger vs. training a new hire.
LoRA / QLoRA	Fine-tune by training only a small add-on and leaving the big model frozen (Q = also squeezed to 4 bits).	Clipping sticky notes onto a textbook instead of rewriting it.
Structured output	Make the model fill a form (JSON that fits a schema) and check it.	A tick-box form, not an essay.
Hallucination & grounding	Models sometimes state things that are not true. Grounding = tie every statement to evidence.	Footnotes. No footnote, no sentence.

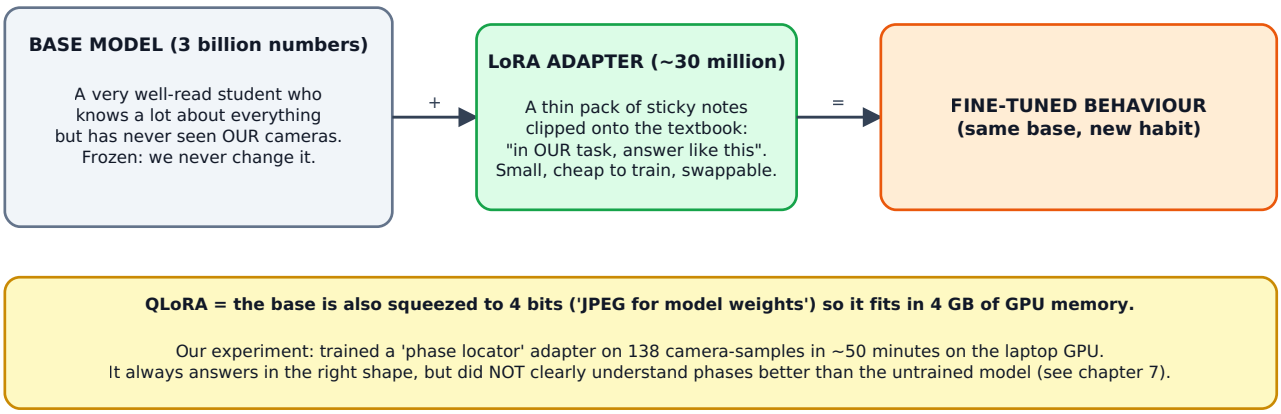


Figure 6.1 — LoRA in one picture, and what our experiment found.

6.1 How we score things (so you can read the results chapter)

Measure	Meaning	Picture
mIoU (phase accuracy)	For each phase, how much the predicted time span overlaps the true span (overlap divided by union), averaged. 1 = perfect, 0 = none.	Two highlighter marks on the same ruler: how much of the ink overlaps?
Boundary error	How many seconds off the predicted phase change is.	How late the referee blew the whistle.
hit@k / R@k	The share of queries for which a correct result appears in the top k.	"Did the right book show up on the first page of results?"
mAP	Average precision: rewards putting all correct results high, not just one.	Not just finding a book, but putting all the right ones at the top.
tIoU@t	Whether the top result's time span overlaps the true span by at least t.	A stricter "right place and right length".
95% interval	A range that says how uncertain a number is, given so few samples. Overlapping intervals mean "cannot tell them apart".	Error bars. Small samples, wide bars.
Paired comparison	Compare two methods clip by clip on the same clips; the clips differ more than the methods do.	Race both runners on the same track, not on different tracks.

7. What the user sees, and who may do what

7.1 The dashboard (React)

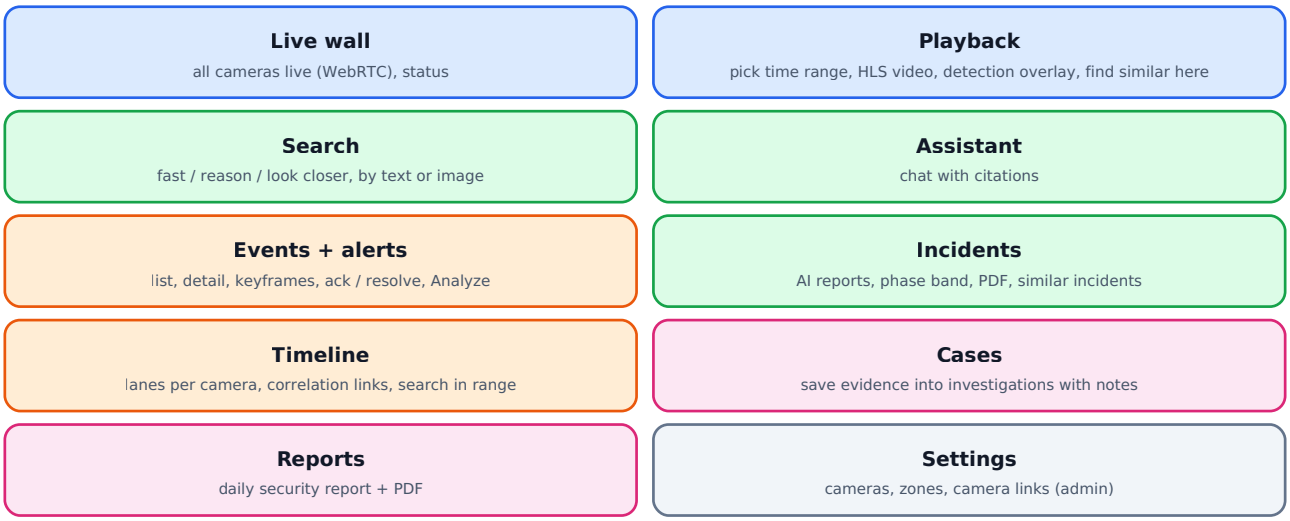


Figure 7.1 — The pages of the web app and what each is for.

The frontend is a **React + Vite + Tailwind + shadcn/ui** single-page app written in **JavaScript (not TypeScript)**, by team decision. Data from the API is validated with **zod** so a wrong shape fails loudly. Each feature lives in its own folder (`features/search`, `features/alerts`, ...) with its API calls, schemas, components, pages and tests. The visual rules (sentence-case text, design tokens, no raw colours) are in `docs/style_guide.md`.

Page	What it does	Good for the demo because...
Live wall	All cameras live over WebRTC, with online/offline status.	shows the system is a real VMS first
Playback	Pick a camera and time range; HLS video; scrub bar with density; detection boxes over the video; "Find similar here".	shows the twin overlay: the AI's view
Search	Type a sentence; choose Fast or Reason; "Look closer"; result cards with why-this-result trace and picture checks; masks; "Search by image".	the headline feature
Events	Verified events and alerts, filters, keyframes, VLM caption, acknowledge / resolve, **Analyze** and live progress.	the operator workflow
Incidents	AI incident reports: phase band, per-camera lanes, evidence frames, PDF, similar past incidents.	the "wow" of GenAI explanation
Assistant	Chat with citation chips and the tools it used.	interactive and easy to explain

Timeline	Lanes, correlation links, drag-select, synced playback, search in range.	the investigation view
Cases	Bookmarks with notes.	closes the investigation story
Reports	Daily security report, PDF.	shows scheduled automation
Settings	Cameras, zones (draw polygons), camera links (topology). Admin only.	how rules and correlation are configured

7.2 The API — the one door

The **API** (FastAPI) is the only thing the browser talks to. It authenticates, checks permissions, forwards search and assistant calls to the retrieval service, signs video links, and pushes live updates. Three ways data reaches the browser:

- **REST** (ordinary request/response) for almost everything;
- **WebSocket** for live pushes: new alerts, job progress, "incident ready", camera status. The push is only a hint; the browser refetches the real state from the database, so a missed push never loses data;
- **SSE** (server-sent events) to stream the assistant's answer as it is written.

Errors always use one envelope `{"error": {"code", "message", "details", "request_id"}}`, which makes the UI's messages consistent.

7.3 Security and roles

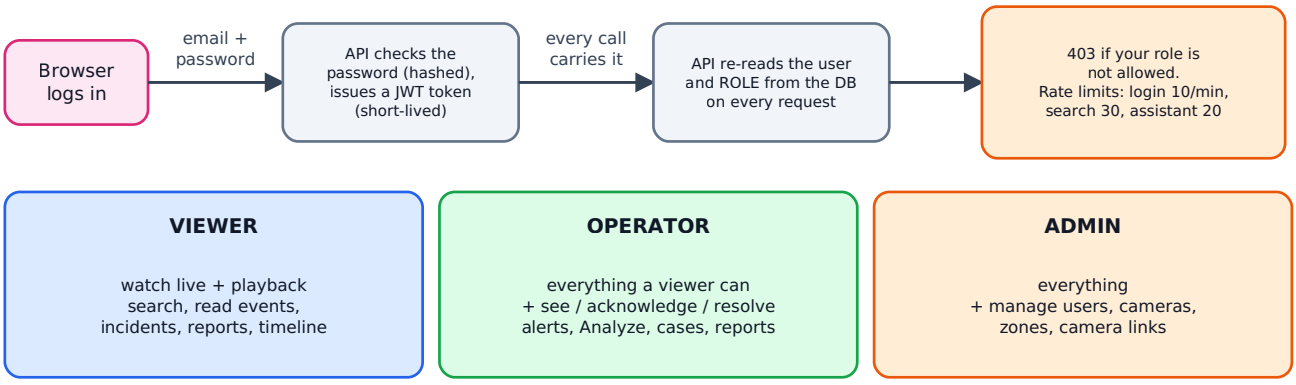


Figure 7.2 — Login, roles, and the main protections.

- **Passwords** are hashed; login returns short-lived **JWT** tokens. The API **re-reads the user and role from the database on every call**, so changing a role or deactivating a user takes effect immediately.
- **Three roles** (admin, operator, viewer). A **role × endpoint test matrix** checks every endpoint against every role, so a new endpoint cannot silently be open to everyone.
- **Rate limits** in Redis: login 10 per minute, search 30, assistant 20, analyses and reports 6.

- **Presigned links** for video and pictures expire in 15 minutes and are created per request; permanent links are never stored.
- CORS allow-list, security headers and a Content-Security-Policy; a secret scanner (gitleaks) passes on the repo; secrets come only from the environment.
- **Deliberately out of scope:** face recognition and face blurring.

8. Running it, keeping it healthy

8.1 Where it runs

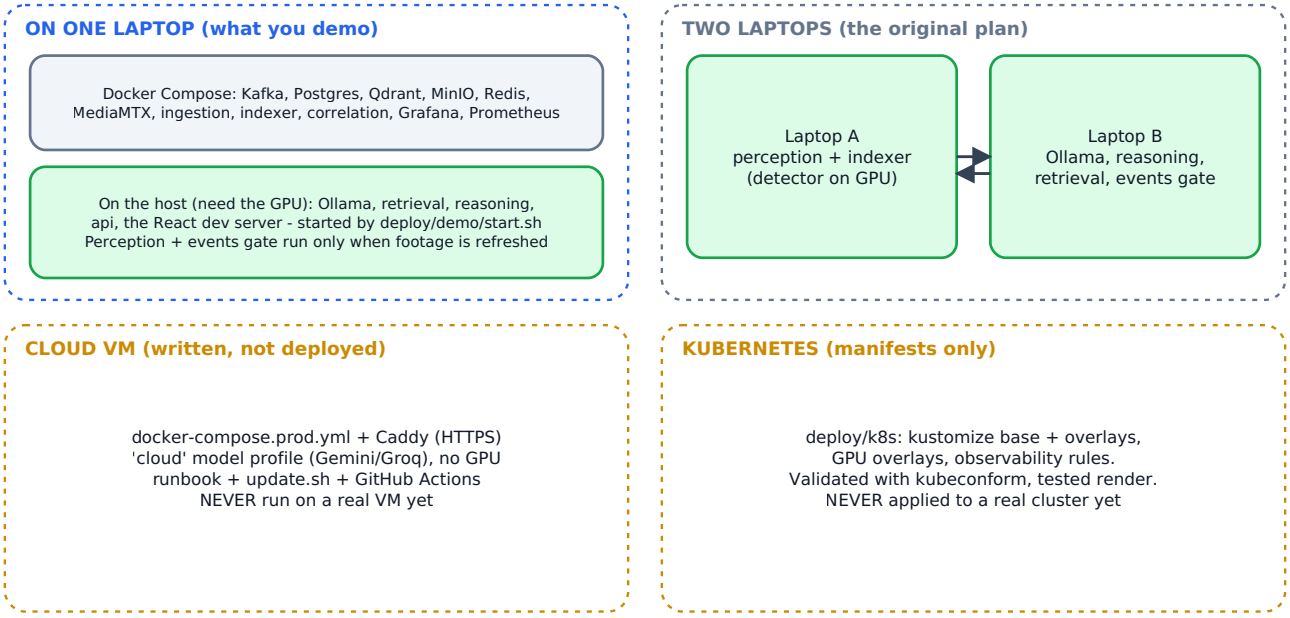


Figure 8.1 — The ways the system can be deployed, and which are proven.

BE HONEST ABOUT…

What is **proven**: everything running on one laptop with Docker Compose plus a few host processes. What is **written but never run for real**: the cloud-VM stack (Caddy, GitHub Actions deployment) and the Kubernetes manifests (they render and pass the validator `kubeconform`, but there was never a cluster to apply them to).

8.2 Commands you may need

```
make setup                    # install Python (uv) + Node deps, pre-commit hooks
make up PROFILE=infra,core    # Kafka, Postgres, Qdrant, MinIO, Redis, MediaMTX, ingestion,
indexer, correlation
make migrate                  # create/upgrade the database tables (Alembic)
deploy/demo/start.sh          # host services: Ollama, retrieval :8010, reasoning, api :8000,
UI :5173
deploy/demo/stop.sh           # stop them again
make demo-refresh             # about 45 min: replay real clips so there is fresh footage to
search
make test                     # all Python unit tests;  make test-int  adds real-database
tests
make lint                     # ruff (Python) + eslint (frontend)
cd frontend && npm run dev    # the UI by itself on :5173
```

FOOTAGE EXPIRES AFTER ONE DAY

The object store deletes video segments and keyframes after 24 hours (a retention policy, to protect disk). Database rows remain, so older search results say "the recording has been removed". **Run `make demo-refresh` the day of your demo.**

8.3 Observability — knowing it is healthy

- Every service exposes **Prometheus metrics** (segments ingested, reconnects, search time per stage, queue depth, Kafka lag, GPU memory, model latency and tokens per provider…).
- A **Grafana dashboard** with 22 panels (lag, dead letters, GPU memory, ingestion per camera, API rate/errors/latency, firing alerts) and nine **alert rules** that are themselves unit-tested with `promtool`.
- Logs are structured JSON with request ids so one request can be followed across services.

8.4 Resilience — what happens when things break

What breaks	What happens
A worker restarts mid-stream (indexer, events, correlation)	Kafka replays from the last commit; idempotent writes avoid duplicates. We restarted them five times in 7 minutes: no segment gap, 839/839 frame points and 1,403/1,403 track points present, no event in two groups.
Ollama (the language models) stops	Fast search is unaffected; reason search falls back to the fused order with a note; the assistant says the model is unavailable; the daily report uses its template; an analysis job fails with a clear reason. Recording and playback never depend on it.
A message is poisoned	After retries it goes to the dead-letter topic; there is a replay tool.
A GPU holder crashes	The lease expires by itself in 30 s.
Redis is down	The cache is a miss, rate limiting fails open; heavy models refuse to run without the lease.

8.5 Testing and CI

- About **2,400 Python unit tests**, about **600 frontend tests**, and several hundred **integration tests** that start real Postgres, Redis, Kafka and S3 containers (testcontainers). Tests never touch the network or a real model: a `FakeGateway` replays recorded answers.
- Many tests were **mutation-checked**: we deliberately broke the code and confirmed a test failed.
- **GitHub Actions** runs path-filtered lint+tests per service, an integration job, frontend lint/test/build, and Docker builds; an aggregate `ci-success` job gates merging.

BE HONEST ABOUT…

CI last ran on GitHub on 8 October. Work since then (this guide's final week) was tested locally but has **not been through CI**, because no pull request was opened.

8.6 The repository and the team

```
genai-vms/
  libs/vms_common/   shared: contracts, Kafka, storage, LLM gateway
  libs/vms_db/       database models + one migration history
  services/          ingestion perception indexer events
                     correlation retrieval reasoning api
  frontend/          React app (JavaScript)
  config/            models.yaml  rules.yaml  correlation.yaml  vqa_bank.yaml
  tools/             camera_sim  annotation_kit
  ml/                annotation  training  evaluation (benchmarks and their results)
  deploy/            compose  k8s  demo
  docs/              PRD  SRS  context  design_architecture  backlog  style_guide  techstack
```

Team. Two builders worked in parallel on separate "tracks" that meet at the message contracts. **Track D (Divyansh)**: ingestion, perception, events, the LLM gateway, text search, reasoning, the assistant, deployment and evaluation. **Track J (Jatin)**: the API, frontend, indexing and playback, correlation, alerts, image search, reports and the timeline. Kuldeep and Pankaj supported documentation, testing and evaluation. Rules in the repo: services never import each other (only shared libs), no TypeScript, no model name in code, no video on Kafka.

9. What we measured

Everything in this chapter was measured on the build laptop: **RTX 3050 4 GB, Ryzen 5 5600H, 14 GB RAM**, local profile. Where a number is a small sample it says so. **Quote only these numbers.**

9.1 Perception and models

Measurement	Result
GPU memory, YOLO11s + SigLIP 2 loaded and running a batch of 8	**966 MB** peak (the budget was 4 GB)
YOLO11s on 8 frames of 1080p	37 ms
`qwen2.5:3b` structured query plan, warm	**0.25 s** (cold start 6.5 s)
`qwen2.5vl:3b` with 1 / 2 / 4 images, warm	3.1 s / 5.4 s / 11.0 s (about 1,050 prompt tokens per image)

9.2 Search latency

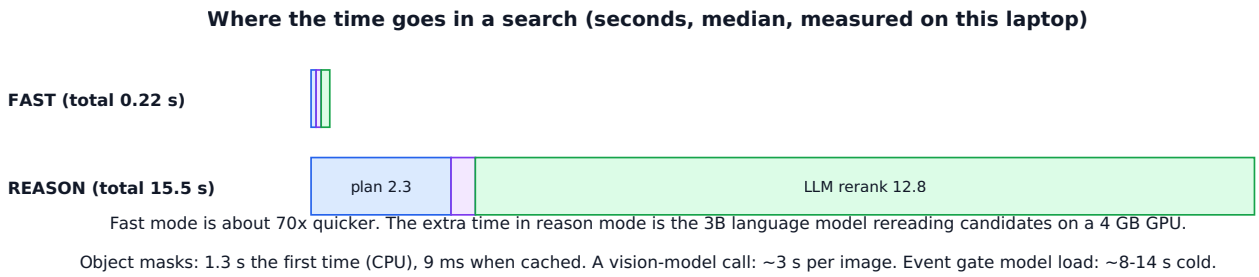


Figure 9.1 — Where search time goes (30 fast and 5 reason runs).

9.3 Real footage through the whole pipeline

Three real bus-station cameras (MEVA), 10 minutes each, with perception on the GPU and the VLM gate on all 49 candidates: **37 verified alerts** in total, the gate rejecting false alarms such as all 9 doorway "intrusions" (see chapter 4). One camera's twin found 127 person tracks, 13 backpack and 5 suitcase tracks.

9.4 Retrieval quality on real labels (MEVA activity clips)

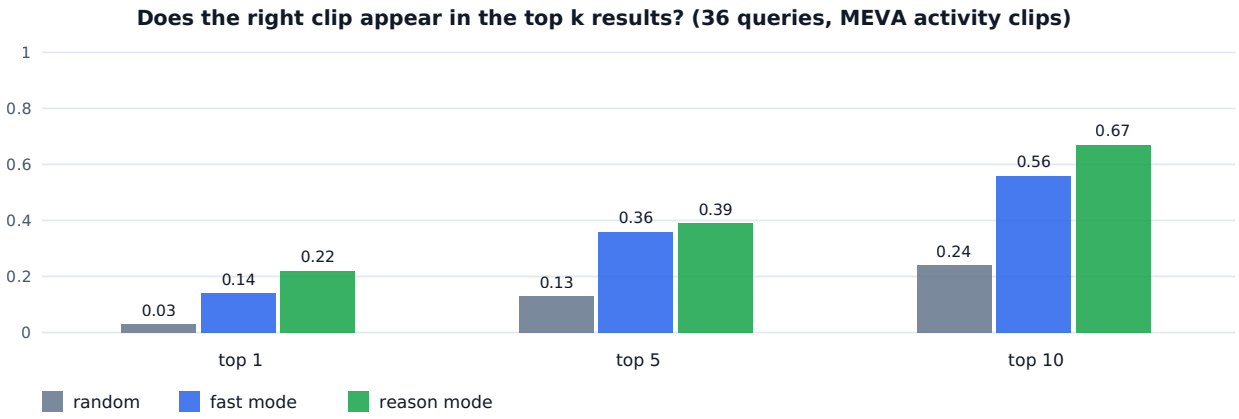


Figure 9.2 — 36 plain-language queries; a result counts if it overlaps a clip of that activity.

Fast mode puts a relevant window at rank 1 for 5 of 36 queries and in the top 10 for 20 (random would be about 1 and 9). Reason mode: 8 and 24, at roughly 70 times the latency; query by query it was better on 12, worse on 7. **What it shows:** search is good at what is in the picture (phones, laptops, bikes, trunks, 7 of 7 in the top 10) and poor at actions that need motion (turning left, handing over, entering through a door).

BE HONEST ABOUT…

The clips are short and staged with black gaps between them, so these numbers are a **ceiling**, not a promise for a day of one real camera. One query per activity, written by the builder. A newer, larger harness (63 queries, explicit vs implicit, four configurations, mAP and tIoU) is built and tested, but **its final clean run did not finish**: the first attempts were invalidated by a recording gap and stale data, which taught us useful lessons (chapter 10).

9.5 The fine-tuning experiment — the honest headline

We trained a QLoRA adapter for the **phase locator** on the real 3B model, on the 4 GB laptop GPU, in about 50 minutes (138 training samples; 26 validation; 32 held-out test samples from 18 clips). We scored it on the held-out set.

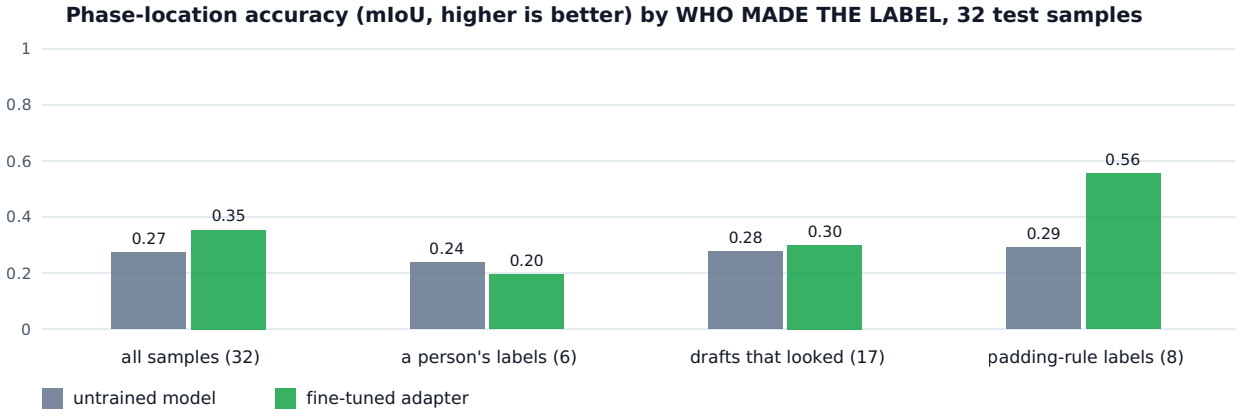


Figure 9.3 — Fine-tuned adapter vs untrained model, split by who made the labels.

- Overall the adapter looks better (0.354 vs 0.274), and the paired difference is small but positive (+0.080, 95% interval +0.005 to +0.153).

- **But the whole gain is on the 8 samples whose labels came from a padding rule** (+0.265): the adapter learned the labelling rule. On a person's labels (6 samples) it was slightly worse (−0.04) and on drafts that looked at the clip it was about equal (+0.02), both with intervals that include zero.
- What it does win: it **always answers in the right form** (32 of 32 usable answers vs 23 of 32), puts boundaries 1 to 2 seconds nearer, and labels more frames correctly (0.61 vs 0.47).
- It stayed well below the detector's own timing on judged labels, and it almost never says "precursor" or "escalation". So we **did not make it the default**.

HOW TO SAY IT

"We proved the pipeline: we can fine-tune a 3B vision model on a 4 GB laptop and serve it. But with labels that are mostly model drafts, we cannot claim it understands incident phases better. A fair test needs more labels made by a person, which is the next step."

BE HONEST ABOUT…

Labels: of 119 labelled clips, a person marked 9 from scratch and checked 20 drafts; 58 are model drafts made from contact sheets, 25 come from a padding rule, 7 from the dataset's crime span. Test samples from one clip (its 2 or 3 camera views) are not independent, so the intervals are tighter than they should be. One run, one seed.

9.6 Assistant and report quality

72 test questions were asked before and after fixes. Objective checks (no model judging): the **first lookup was the right tool** 78% → 100%; the lookup used the **camera asked about** 28% → 100%; used the **day asked about** 0% → 100%; tags that matched no record fell from 27/92 to 14/80. The automated judge (a 3B model, a different family from the generator) gave high but unreliable scores — it named a "problem" in every single answer — so **we do not claim answer quality from it**; a 20% human-verification sheet exists but was not filled in.

9.7 Tests and delivery

Roughly three quarters of the planned backlog (by points) is done and ticked with evidence. Two thousand four hundred Python unit tests, about six hundred frontend tests and the integration suites passed at the last full run (27 October).

10. What is not done, and what went wrong

Examiners trust a presenter who names the gaps first. This chapter is your list. Say these calmly: they are normal for a project built in weeks.

10.1 Not done

Gap	Why / status
Fine-tuned adapter does not beat the untrained model on judged labels	Labels are mostly model drafts. Result recorded honestly (9.5).
Second adapter (PhaVR: captions and Q&A) not trained	Needs caption annotations; the three stories that depend on it stay open.
Cloud-VM deployment and Kubernetes never run for real	Written and validated, but no VM or cluster was available. No domain, no certificates.
Four-way retrieval ablation: final clean run not completed	Harness, 63 queries and metrics are built and tested; the recorded stream expired before a clean run finished.
Usability study (SUS) and human scoring sheets	Kit is written; no sessions were held.
Long load tests (2/4/6 cameras for 30 minutes per profile)	Shorter real runs exist; one real camera feed plus replays.
Hybrid / cloud model profiles never exercised with real keys	No provider keys on the build machine.
Find-similar, timeline links and search-in-range clicked through in a real browser	Built and unit-tested in a simulated browser; a live browser pass needs fresh footage.
Daily report indexed into the knowledge search	Not built; the assistant reads reports straight from the database.
CI on the newest commits	No pull request was opened.
Guide's approval of the phase scheme	Not needed (confirmed by you).

10.2 Things we found and fixed late (good stories to tell)

Benchmarks and real runs are how these were discovered, which is the point of benchmarking.

What we found	How	Fix
Track ids were reused after a restart with no checkpoint, so a second recording of a camera kept only 9 of about 2,800 tracks in the database.	The retrieval benchmark's timeline alignment failed.	Track numbers now start at the clock in milliseconds when there is no checkpoint. Unit-tested.

A stuck GPU-lease holder blocked every vision call until the service was restarted.	"Look closer" never produced an answer in a benchmark.	Holders now stop being renewed after 15 minutes. Tested.
The training run ran out of GPU memory on 4 GB.	First real training attempt.	Five changes (matching weight type, 8-bit optimizer, no dropout, offloading saved activations to host memory, freeing cache); peak 3.3 of 3.68 GB.
Image size mismatch: the serving config shrank frames the adapter was trained on.	Reviewing the evaluation settings.	Serve at 640 px; both settings are reported.
A **4-minute ingestion stall** under memory pressure left a hole in a recorded stream.	Segment-gap check before a benchmark.	Re-recorded; the benchmark now searches only inside the recording under test.
The **database migration** for the search log was not applied on the live database.	Search log inserts failed quietly in logs.	Migration applied.
The disk was 96% full.	Routine check.	Removed rebuildable caches and unused images (about 10 GB).

WHAT TO SAY

"Our benchmarks found real bugs in our own system — ID reuse, a lease that could wedge the GPU — and we fixed them with tests. I'd rather show you that than a perfect-looking number."

11. How to present it

11.1 The shape of a good talk



The story arc: WATCH -> FIND -> UNDERSTAND -> EXPLAIN -> ASK -> INVESTIGATE

Figure 11.1 — The story arc to follow in a talk and in the demo.

1. **Hook** (problem): one person, many screens, hours of video.
2. **Idea**: turn video into structured knowledge once; reuse it three ways.
3. **Show the machine**: the big-picture figure (figure 2.1) and **one clip's journey** (figure 3.1).
4. **Show it working**: the live demo.
5. **Show it is honest**: results with their limits.
6. **Close**: what you would do next.

11.2 The 1-minute pitch

> **SAY THIS**
> "Surveillance cameras record far more video than anyone can watch. Our system turns every ten-second clip into a structured report card — who and what is there, where, and doing what — and then uses that to do three things. First, you can **search the footage in plain words**. Second, it **detects events with simple rules, has a vision AI double-check them, and links the same event across cameras**. Third, it **writes an incident report in phases, where every sentence cites its evidence**, and you can chat with an assistant about it. It runs locally on a 4 GB GPU laptop, and we measured it honestly, including where it falls short."

11.3 The 5-minute talk (7 slides)

#	Slide	Say (about)	Time
1	**Problem** (figure 1.1)	Cameras produce hours of video nobody can search. A person scrubbing through footage is slow and misses things.	0:40
2	**Idea: the digital twin** (figure 4.3)	We convert each 10-second clip into a small structured JSON (objects, tracks, attributes, zones, vectors). One representation feeds search, events and reports.	0:45
3	**Architecture** (figure 2.1)	Four stages: ingest, understand, notice and check, explain. Kafka between services, heavy files in object storage, one gateway for every model, a GPU lease so a 4 GB card works.	1:00

4	**Cheap then expensive** (figures 4.5 and 5.2)	Rules first, vision AI second; embeddings first, language-model rerank second. That is why it runs on a laptop.	0:45
5	**Live demo** (chapter 12, short form)	Search fast vs reason; an alert; Analyze to an incident report with citations.	1:00
6	**Results, honestly** (figures 9.2 and 9.3)	Fast search 0.22 s; finds what is in the picture, weak on actions. Our fine-tuned model learned the format but not yet the understanding — we say so.	0:40
7	**Next steps**	More human labels, train the second adapter, re-identification across cameras, a real deployment and user study.	0:10

11.4 The 15-minute talk

Min	Part	Content
0–2	Problem and the three pillars	Chapter 1; the one-minute pitch.
2–5	Architecture	Figure 2.1, then one clip's journey (figure 3.1). Mention Kafka, MinIO, Postgres, Qdrant, Redis in one sentence each using the analogies.
5–7	Perception and events	Twin (figure 4.3); six rules; the VLM gate and the real example (30/30 loitering verified, 9/9 doorway intrusions rejected); correlation.
7–9	GenAI: search and reasoning	Figures 5.2 and 5.5; evidence-or-it-didn't-happen; provenance.
9–12	Live demo	Chapter 12 in full.
12–14	Results and limits	Latency; retrieval chart; the adapter experiment and its honest reading; the bugs the benchmarks found.
14–15	Next steps and questions	Chapter 10's gaps framed as the roadmap.

11.5 Presentation habits that help

- **Tell the story of one clip** rather than listing components.
- **Know five numbers** by heart: **10 s** segments; **4 GB** GPU; **0.22 s** fast search; **~15 s** reason search; **966 MB** perception memory.
- **Name a limit before they find it.** "One thing I want to be clear about..."
- When you do not know, say "I measured X; I did not measure Y". That is a strong answer.
- Use analogies: post office (Kafka), warehouse / filing cabinet / meaning map (MinIO / Postgres / Qdrant), detective's report card (twin), one chair (GPU lease), footnotes (citations).

12. The live demo, step by step

12.1 Preparation checklist

When	Do	Why
The day before	`make demo-refresh` (about 45 minutes). It replays real clips through the pipeline, lets the rules and VLM gate judge them, and restarts the demo stack.	Footage in storage expires after 24 hours. Without it, search shows "recording removed".
1 hour before	`make up PROFILE=infra,core`; `make migrate`; `deploy/demo/start.sh`.	Containers + the host services (Ollama, retrieval, reasoning, API, UI).
30 minutes before	Stop perception and the camera simulator. Check `curl localhost:11434/api/ps` shows the model on the GPU (`size_vram` not tiny).	Perception, the gate and the language model cannot share a 4 GB card; if Ollama falls back to CPU, reason mode becomes very slow.
10 minutes before	Log in (admin account from `.env`) at `http://127.0.0.1:5173`. Open Search once so the encoder is warm; run one reason search so the language model is loaded.	The first call is a cold start (8 to 14 s).
Close other things	Close extra browser tabs and heavy apps.	14 GB RAM is shared with the models; memory pressure caused slow runs before.

12.2 The script

#	Click	Say	What you should see
1	**Live wall**	"These are the cameras, live over WebRTC. This is a normal VMS. Everything after this is what the AI adds."	Camera tiles with status.
2	**Playback** → pick a camera and range → Load → turn Detections on	"The coloured boxes are the digital twin: objects the AI found and tracked. The video is raw; this is what the system understood."	Video with boxes and ids; scrub bar.
3	**Search** → type "a person in a red top near the service door" (or any example shown) → **Fast**	"Fast mode: picture similarity only, about a quarter of a second."	Result cards with keyframes and a match percentage.
4	Same query → **Reason**	"Now a language model re-reads the best candidates against the system's records. It takes about fifteen seconds on this laptop, and each card tells you why it ranked there."	Re-ranked cards with a "Why this result" box and a model score.

5	Click **Find similar** on a card (if time allows), draw a box around a person	"I can also draw a box on a frame and search for things that look like it."	A dialog with similar results. (Built; browser-tested only in simulation, so use it last.)
6	**Events** → open one	"A rule noticed this, then a vision model looked at the keyframes and confirmed it, with this caption. The operator can acknowledge it. If the model had said 'no', the operator would never have seen it."	Event detail with keyframes, caption, verdict.
7	Click **Analyze**	"Now the reasoning service finds the five phases, reads each camera, and writes a report."	A progress bar with stages (reading action · cam02…). Takes two to four minutes on this laptop.
8	**Incidents** → open the report	"Every sentence cites evidence ids; click one to see the frame. If the model cannot cite, the sentence is dropped. Here is the PDF. And at the bottom, which model produced it."	Phase band, lanes, cited evidence frames, PDF button.
9	**Assistant** → ask "What happened today?", then "How many people were on <camera> today?"	"It can only call seven fixed tools. The first line is the tool's own sentence; the citations are real records."	Streaming answer, tool trace, citation chips.
10	**Timeline** → drag across a lane	"One lane per camera. Events linked across cameras are outlined. I can play cameras together, search just this range, or save it to a case."	Lanes, Linked row, selection bar.
11	**Reports**	"A daily report is produced automatically; every number in the text is checked against the figures."	Report page with charts and PDF.

12.3 If something goes wrong

Problem	Say / do
Reason search takes much longer than 15 s	"It is a 3B model on a 4 GB card; if it is slow the system falls back to the fast order and says so." Check `/api/ps` for GPU use afterwards.
Search shows "recording removed"	The footage expired. Explain the retention policy and show the report pages (they keep their evidence frames).
Analyze fails or hangs	Show an existing incident report. Explain that jobs show a clear failure reason and the system never half-writes a report.
The language model is down	Show that fast search, live view and playback still work: "degrade, don't die".

13. Questions you will be asked

Short, honest answers. Adapt them to your own voice.

1. Why not just send the video to a big cloud model?
Cost, privacy and volume. Hours of video from many cameras is far too large to send; the data is sensitive; and cloud models have rate limits. Our design converts video to small structured text once, locally, and uses cloud models only as an option for text reasoning (the hybrid profile).

2. What is a digital twin here?
A small structured JSON for each 10-second segment: objects, tracks, attributes, zones, motion and vectors. It is a text copy of what the video contains, so search, rules and reports read it instead of re-watching video.

3. Why Kafka?
It decouples fast and slow parts. The slow GPU service never blocks ingestion; a crashed service resumes from where it stopped; messages can be replayed. Messages carry links, not video.

4. How do you stop the AI making things up?
Structured outputs checked against a schema, retries that quote the errors, and **mandatory citations**: every claim must point to an evidence id that exists, and uncited claims are dropped. Also fixed tools for the assistant, numeric post-checks for the daily report, and provenance on every result.

5. Why both rules and an AI gate?
Rules are cheap and run on everything but produce false alarms; the vision model is accurate but slow and uses the GPU, so it only checks the suspicions. On real footage the gate rejected all 9 doorway "intrusions" and kept 30 of 30 loitering events.

6. How does it run on only 4 GB?
Small models (3B) squeezed to 4 bits, one model family on the GPU at a time through a lease, embeddings on the CPU, masks on the CPU on demand, and a "cheap before expensive" design. Perception needs under 1 GB.

7. What is the GPU lease?
A lock in Redis that says which model family currently occupies the GPU. Others wait their turn; it expires if the holder crashes (and after 15 minutes even if stuck).

8. What is the difference between fast and reason search?
Fast = picture and text similarity, about 0.2 s. Reason = a language model decomposes the query and re-reads the best candidates (and optionally a vision model checks details), about 15 s. Reason mode is the "reasoning-based" part of the title.

9. Did the fine-tuning work?
Technically yes: we trained a 3B vision model on a 4 GB GPU and served it. As an accuracy claim, not proven: the gain was only on labels made by a rule; on a person's labels there was no gain. We report that and we did not make it the default.

10. How do you know search works?
We built a benchmark from real MEVA labels (36 queries): fast mode finds a relevant clip in the top 10 for 20 of 36 queries (random would be about 9), reason mode 24. It is strong on objects and postures, weak on motion. The clips are staged, so it is a ceiling.

11. What does idempotent mean, and why do you care?
Doing an operation twice gives the same result as once. Kafka can redeliver messages, so every consumer writes idempotently to avoid duplicates after a restart.

12. How does correlation work without recognising people?
It uses time and a camera map: two events on connected cameras whose times fit are linked, and chains are merged with union-find. The limit is that it cannot tell whether it is the same person; re-identification is future work.

13. What happens if the language model is down?
Recording, live view, playback and fast search continue. Reason search falls back with a note, the assistant says the model is unavailable, the daily report uses a template, and an analysis job fails with a clear reason.

14. How is it secured?
Hashed passwords, short-lived JWTs, roles re-read from the database every call, a role-by-endpoint test matrix, rate limits, 15-minute presigned links, audit logging, no secrets in code. Face recognition is deliberately not built.

15. What is novel compared with the papers?
One shared digital twin feeding search, reasoning and reports; running on continuous live streams, not short clips; running on a 4 GB laptop; a unified operator experience with evidence-cited reports.

16. What did you personally build?
(Track D) ingestion, perception, the events service and the VLM gate, the LLM gateway and GPU lease, text search, reasoning and incident reports, the assistant, the deployment and the evaluations. Jatin built the API, frontend, correlation, alerts, indexing and the timeline. Several cross-track pieces were done deliberately and flagged.

17. Why Qdrant and Postgres?
Postgres for exact relational facts; Qdrant for similarity search with payload filters and hybrid dense+sparse search. Each does its job well.

18. Why JavaScript and not TypeScript?
A team decision based on skills. We compensate with runtime validation (zod) at the API boundary and tests.

19. How would you scale it?
Kafka partitions (6 per topic) let consumers scale out; the indexer can scale horizontally; heavy models would move to a bigger GPU or the cloud profile. The tested target here is 4 to 6 cameras.

20. What are the biggest limitations?
Small models, so quality is limited; search is weak on actions that need motion; the fine-tuning experiment was inconclusive because labels were mostly drafts; cloud and Kubernetes deployments were not exercised on real infrastructure; no user study.

21. What would you do next?
Collect more human-made labels and retest the adapter on them; train the second adapter; add person re-identification; finish the clean retrieval ablation; deploy to a VM and run the usability study.

22. What are MEVA and UCF-Crime?
Public research video datasets: MEVA has multi-camera everyday activity (we used bus-station cameras and activity clips); UCF-Crime has crime videos with labelled categories. We used them as stand-ins for live cameras.

23. What is MinIO / S3?
An object store: a place for big files, each with a link. S3 is Amazon's version; MinIO is a free one with the same interface, so the project can later switch to real S3.

24. How is the system tested if models are non-deterministic?
Unit tests never call a real model: a `FakeGateway` replays recorded answers; golden tests run the report writer on real stored evidence bundles; benchmarks measure the real models separately.

25. Why 10-second segments?
Short enough for low latency and cheap re-processing, long enough to track an object and keep file counts sane. It is a setting.

26. Is it legal / ethical to monitor people?
We use public research datasets and team members for demos, keep data local by default, avoid face recognition, log access, and expire footage. Real deployments need consent, signage and policy decisions that are not ours to make.

27. What is the "gate" rejecting?
Candidates where the vision model answers "no" (or "unsure" for a rule that does not accept it). Rejected ones are stored with their reason, never shown to operators, and useful for measuring the gate.

28. What does "provenance" mean?
The record of how a result was made: which model, how many tries, whether a fallback was used, how long each step took. It lets you audit and compare.

29. What if I ask for something that never happened?
Search returns the closest matches with scores (it cannot say "none exist" for a visual query, but the reason mode's trace and low scores show weak matches). The assistant says it found nothing for questions about records.

30. How long did it take, and how big is it?
About six weeks of two-person work (kickoff 17 September); roughly 74 stories planned, about three quarters finished and tested; thousands of automated tests across eight services, a shared library and a frontend.

14. Glossary and cheat sheet

14.1 Glossary

Term	Meaning
Adapter (LoRA)	A small add-on trained on top of a frozen model to change its behaviour for a task.
Alert	A notification created from a verified event; lifecycle open → acknowledged → resolved.
API	The service the browser talks to; checks login and roles.
BM25	A classic keyword-ranking method; the "exact words" half of hybrid search.
ByteTrack	The tracker that keeps ids stable across frames.
Candidate	A rule hit before the vision model has checked it.
Contract	An agreed message format (Pydantic class + sample file + version) between services.
Correlation group	Verified events on linked cameras, close in time, treated as one story.
Dead-letter topic (DLQ)	Where messages that keep failing are parked.
Digital twin	Structured JSON describing what a segment contains.
Embedding	A vector of numbers capturing the meaning/appearance of a text or image.
Evidence id	A stable id (like `ev-cap-03`) for a caption or answer that reports must cite.
Fallback	The next model (or method) tried when one fails.
Gateway	The single place every AI call passes through (cache, lease, validation, provenance).
GPU lease	The Redis lock that lets one model family use the GPU at a time.
HLS / WebRTC	Two ways to play video in a browser: HLS for playback, WebRTC for low-delay live view.
Idempotent	Safe to repeat: same result the second time.
JIT refinement	"Just in time": ask a vision model about a detail only when a result needs it.
JWT	A signed login token the browser sends with every request.
Kafka	The message system between services.
Keyframe	One saved picture per second of video.
LLM / VLM	Large language model / vision-language model.
MediaMTX	The media server that re-broadcasts camera streams.
MinIO	S3-compatible object storage for large files.

Ollama	A program that runs language and vision models locally.
Phase	One of baseline, precursor, escalation, action, aftermath.
PhaVR / TG	The two reasoning tasks: reading each phase (captions + Q&A), and locating the phases in time (temporal grounding).
Postgres	The relational database.
Profile	local, hybrid or cloud: which models serve the tasks.
Provenance	The record of which model and steps produced a result.
Qdrant	The vector database.
RAG	Retrieval-augmented generation: look things up first, then write the answer from what was found.
Redis	The fast scratchpad: lease, cache, push, limits.
RRF	Reciprocal rank fusion: merge rankings by summing 1/(60+rank).
SAM 2	A model that draws precise object outlines (masks).
Segment	A 10-second recorded piece of one camera's stream.
SigLIP 2	The model that makes embeddings for both text and images.
Topology	The camera map: overlap and transit links.
Track	One object followed over time (`cam03-t412`).
VQA bank	A list of standard questions per event type asked of each phase's frames.
WebSocket / SSE	Ways the server pushes live data to the browser.
YOLO11	The object detector.
Zero-shot	Using a model without any task-specific training.
Zone	A polygon drawn on a camera view (entrance, restricted area...).

14.2 Numbers and facts to memorise

Fact	Value
Segment length	10 s; one keyframe per second
Target scale	4 to 6 cameras on a 4 GB GPU
Perception GPU memory (YOLO11 + SigLIP 2)	966 MB peak
Models (local profile)	`qwen2.5vl:3b` (vision), `qwen2.5:3b` (text), via Ollama
Fast / reason search	0.22 s / 15.5 s median
Search windows	hits within 10 s on a camera merge; top 30 kept; rerank reads the top 10 (5 per call)
Blend in reason mode	40% fused score + 60% model score

Rules	6: two intrusion rules, loitering (60 s), crowding (more than 8 for 20 s), abandoned object (30 s + 20 s), running
Phases	baseline, precursor, escalation, action, aftermath
Kafka topics	segments, twin, events, correlations, incidents, dlq (all `.v1`)
Assistant	7 tools, at most 4 lookups per question, last 12 messages kept
Roles	admin, operator, viewer
Presigned link lifetime	15 minutes
Footage retention	24 hours in object storage
Ports	UI 5173, API 8000, retrieval 8010, Ollama 11434, MinIO 9000, Qdrant 6333
Adapter training	QLoRA r=16, 138 samples, 3 epochs, about 50 minutes, 3.3 GB peak GPU
Retrieval benchmark (Oct 6, 36 queries)	hit@1/5/10: fast .14/.36/.56; reason .22/.39/.67; random .03/.13/.24
Adapter vs untrained (mIoU)	0.354 vs 0.274 overall, but gain only on rule-made labels

THE THREE SENTENCES TO REMEMBER IF YOU FORGET EVERYTHING ELSE

1. **Turn video into structured knowledge once** (the digital twin), then reuse it for search, events and reports.
2. **Cheap filter first, expensive reasoning second**, so it runs on a 4 GB laptop.
3. **Evidence or it didn't happen**: every generated claim cites its evidence, and we report our results honestly, including what did not work.